



Algorithmik – WS 26/27

Gelesen von Prof. Dr. Daniel Gaida

26.08.2026

Prof. Dr. Daniel Gaida

Professor für Cyber-Physische Systeme

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Seite 1

Technology
Arts Sciences
TH Köln

Lernraum II: Höhere Abstrakte Datentypen und Datenstrukturen

- Abstrakte Datentypen
 - Prioritätswarteschlange
- Datenstrukturen
 - Hashtabelle
 - Binäre Suchbäume
 - AVL-Baum
 - Heap

Inhalt: Prioritätswarteschlange / Heaps

- ADT: Prioritätswarteschlange (Vorrangwarteschlange)
- Prioritätswarteschlange Implementierungen mit aktuell bekannten Datenstrukturen
- Binärer Heap: Idee + Invarianten
- Binärer Heap: Methoden
- Prioritätswarteschlange implementiert als binärer Heap

Prioritätswarteschlange: ein neuer ADT

Sie verwalten eine Warteschlange von Essensbestellungen in einem Restaurant, das normalerweise die Bestellungen in der Reihenfolge des Eingangs entgegennimmt (First-In-First-Out).

Plötzlich betritt Lukas Podolski das Restaurant!

Sie möchten ihn und weitere Prominente so schnell wie möglich bedienen. Ihr neues Speisenverwaltungssystem sollte eine Rangfolge der Kunden aufstellen und uns darüber informieren, welche Speisenbestellung wir als Nächstes bearbeiten sollten (die, die höchste Priorität hat).

Prioritätswarteschlange als ADT

Min-Prioritätswarteschlange ADT

state

Menge von vergleichbaren Schlüsseln -
Geordnet nach „Priorität“

behavior

add(key, value) – add a new key-value-pair to the collection

removeMin() – returns the key-value-pair with the smallest priority, removes it from the collection

peekMin() – find, but do not remove the key-value-pair with the smallest priority

Anwendungen:

- Prioritätsgetriebene Warteschlange im Restaurant
- Netzwerkdrucker
- Huffman-Kodierung
- Graph-Algorithmen
 - Algorithmus von Dijkstra

Prioritätswarteschlange - Beispiel

Schlüssel	Wert
0	Bestellung von Lukas Podolski
2	Bestellung von Melanie Musterfrau
10	Bestellung von Max Mustermann
...	

Prioritätswarteschlange als ADT

Wenn eine Warteschlange „First-In-First-Out“ (FIFO) ist, sind Prioritätswarteschlangen „Most-Important-Out-First“.

Schlüssel in der Prioritätswarteschlange müssen vergleichbar sein

Die implementierende Datenstruktur wird ein gewisses Maß an interner Sortierung enthalten, ähnlich wie bei BST/AVL-Baum.



Min-Prioritätswarteschlange ADT

state

Set of comparable keys
- Ordered based on
“priority”

behavior

removeMin() – returns the k-v-pair with the smallest priority, removes it from the collection

peekMin() – find, but do not remove the k-v-pair with the smallest priority

add(key, value) – add a new k-v-pair to the collection

Max Priority Queue ADT

state

Set of comparable keys
- Ordered based on
“priority”

behavior

removeMax() – returns the k-v-pair with the largest priority, removes it from the collection

peekMax() – find, but do not remove the k-v-pair with the largest priority

add(key, value) – add a new k-v-pair to the collection

Implementierung von Prioritätswarteschlangen: Versuch I

Vielleicht wissen wir bereits, wie man eine Prioritätswarteschlange implementiert.

Welche Laufzeit haben `add()`, `removeMin()` und `peekMin()` bei diesen Datenstrukturen?

Hier sind nur
die Schlüssel
dargestellt

Implementierung	add	removeMin	peekMin
Unsortiertes Array			
Verkettete Liste (sortiert)			
AVL-Baum			

3	1	5	4		
---	---	---	---	--	--

Gehen Sie bei Array-Implementierungen davon aus, dass Sie die Arraygröße nicht ändern müssen.

Abgesehen von dieser Annahme sollten Sie eine Worst-Case-Analyse durchführen.

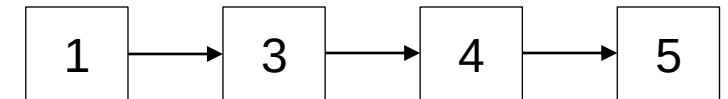
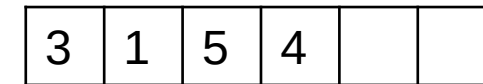
Implementierung von Prioritätswarteschlangen: Versuch I

Vielleicht wissen wir bereits, wie man eine Prioritätswarteschlange implementiert.

Welche Laufzeit haben `add()`, `removeMin()` und `peekMin()` bei diesen Datenstrukturen?

Hier sind nur
die Schlüssel
dargestellt

Implementierung	add	removeMin	peekMin
Unsortiertes Array	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Verkettete Liste (sortiert)			
AVL-Baum			



Gehen Sie bei Array-Implementierungen davon aus, dass Sie die Arraygröße nicht ändern müssen.

Abgesehen von dieser Annahme sollten Sie eine Worst-Case-Analyse durchführen.

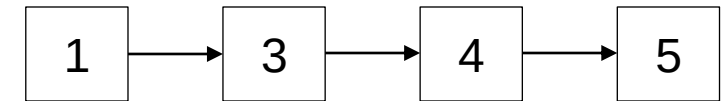
Implementierung von Prioritätswarteschlangen: Versuch I

Vielleicht wissen wir bereits, wie man eine Prioritätswarteschlange implementiert.

Welche Laufzeit haben `add()`, `removeMin()` und `peekMin()` bei diesen Datenstrukturen?

Hier sind nur
die Schlüssel
dargestellt

Implementierung	add	removeMin	peekMin
Unsortiertes Array	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Verkettete Liste (sortiert)	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
AVL-Baum			



Gehen Sie bei Array-Implementierungen davon aus, dass Sie die Arraygröße nicht ändern müssen.

Abgesehen von dieser Annahme sollten Sie eine Worst-Case-Analyse durchführen.

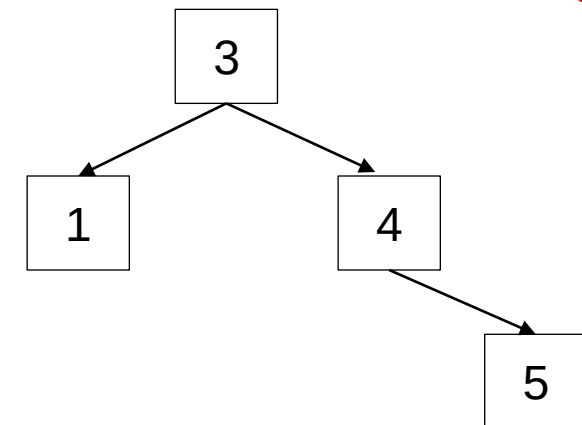
Implementierung von Prioritätswarteschlangen: Versuch I

Vielleicht wissen wir bereits, wie man eine Prioritätswarteschlange implementiert.

Welche Laufzeit haben `add()`, `removeMin()` und `peekMin()` bei diesen Datenstrukturen?

Implementierung	add	removeMin	peekMin
Unsortiertes Array	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Verkettete Liste (sortiert)	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
AVL-Baum			

Hier sind nur
die Schlüssel
dargestellt



Gehen Sie bei Array-Implementierungen davon aus, dass Sie die Arraygröße nicht ändern müssen.
Abgesehen von dieser Annahme sollten Sie eine Worst-Case-Analyse durchführen.

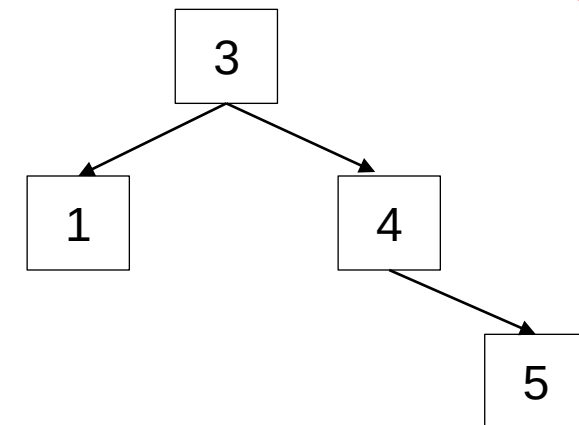
Implementierung von Prioritätswarteschlangen: Versuch I

Vielleicht wissen wir bereits, wie man eine Prioritätswarteschlange implementiert.

Welche Laufzeit haben `add()`, `removeMin()` und `peekMin()` bei diesen Datenstrukturen?

Hier sind nur
die Schlüssel
dargestellt

Implementierung	add	removeMin	peekMin
Unsortiertes Array	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Verkettete Liste (sortiert)	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
AVL-Baum	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$



Gehen Sie bei Array-Implementierungen davon aus, dass Sie die Arraygröße nicht ändern müssen.
Abgesehen von dieser Annahme sollten Sie eine Worst-Case-Analyse durchführen.

Implementierung von Prioritätswarteschlangen: Versuch I

Vielleicht wissen wir bereits, wie man eine Prioritätswarteschlange implementiert.

Welche Laufzeit haben `add()`, `removeMin()` und `peekMin()` bei diesen Datenstrukturen?

Implementierung	add	removeMin	peekMin
Unsortiertes Array	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Verkettete Liste (sortiert)	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
AVL-Baum	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$

AVL-Bäume sind unsere Basis – wir schauen uns heute Heaps als alternative Datenstruktur an und kommen später auf AVL-Bäume als Option zurück

Laufzeiten für **`peekMin()`** wären konstant, wenn Sie ein Feld hinzufügen, um das kleinste Schlüssel-Wert-Paar zu speichern. Bei jedem Einfügen und Entfernen Feld aktualisieren.

Fragen?

Dinge, über die wir gesprochen haben:

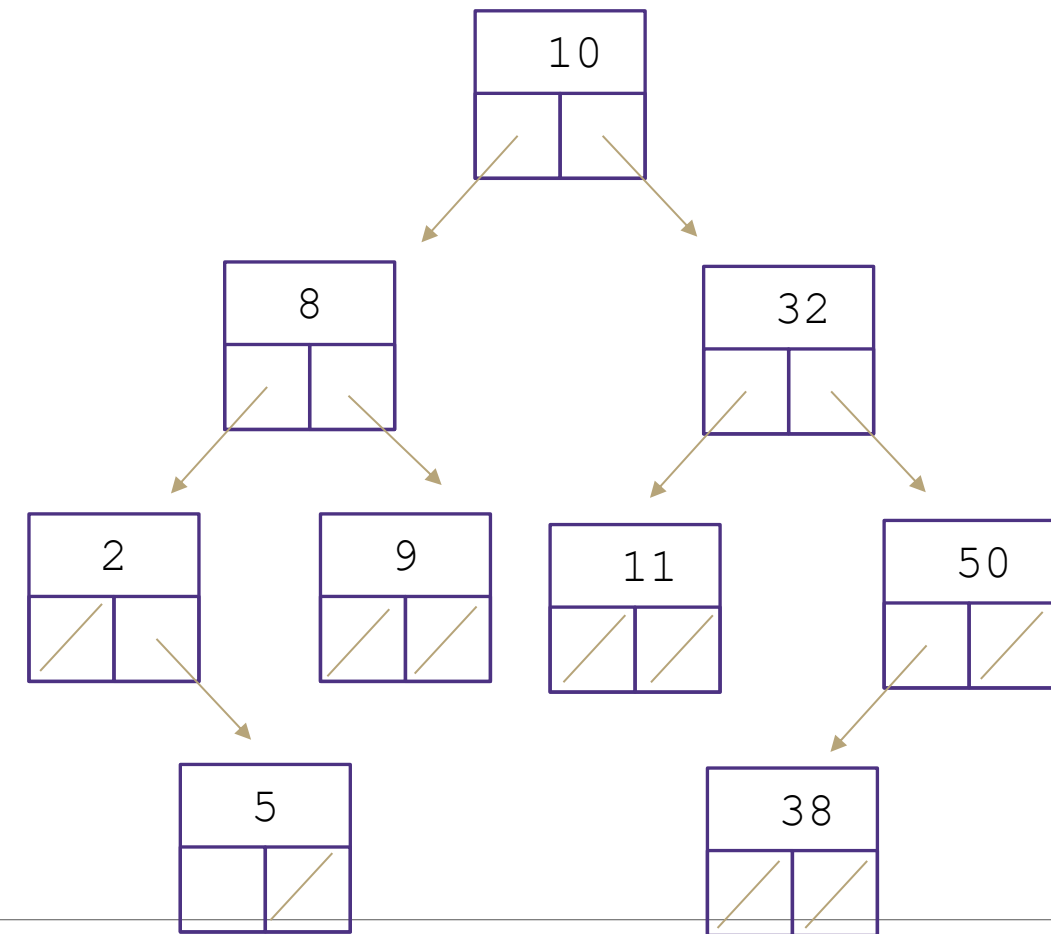
- Prioritätswarteschlange ADT
- Mögliche Implementierungen der Prioritätswarteschlange

Zur Erinnerung: Binärer Suchbaum

Ein binärer Suchbaum ist ein binärer Baum mit folgender Invariante:

- Für jeden Knoten mit Schlüssel k :
 - Der linke Teilbaum hat nur Schlüssel kleiner als k
 - Der rechte Teilbaum hat nur Schlüssel größer als k

Die Zahlen in den Knoten sind die Schlüssel von Schlüssel-Wert-Paaren



Zur Erinnerung: Die BST-Invariante gilt rekursiv für den gesamten Teilbaum

Heaps

Die Idee:

Die BST-Invariante sorgt dafür, dass wir einen kleineren Schlüssel finden, wenn wir in den linken Teilbaum gehen und einen größeren Schlüssel, wenn wir in den rechten Teilbaum gehen.

Jetzt wollen wir nur den kleinsten Schlüssel schnell finden, also schreiben wir eine andere Invariante:

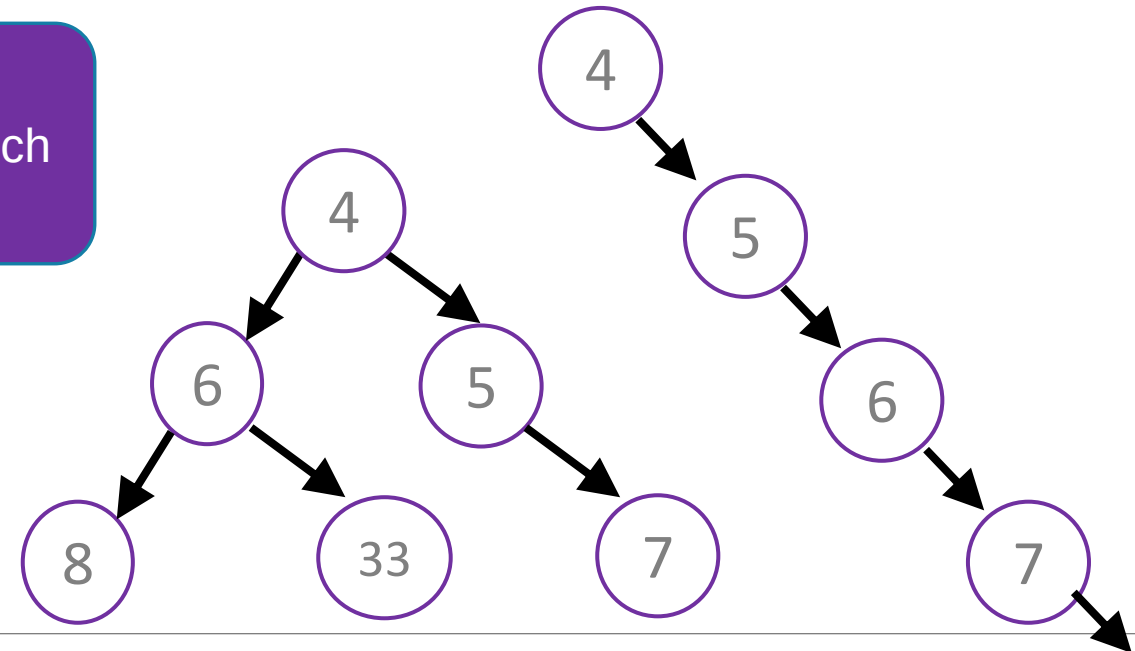
Min-Heap-Invariante

Der Schlüssel jedes Knotens ist kleiner als oder gleich der seiner Kinder.

Der kleinste Schlüssel befindet sich an der Wurzel!

Jetzt ist `peekMin()` super einfach!

Brauchen wir noch mehr Invarianten?



Heaps

Mit der aktuellen Definition könnten wir immer noch degenerierte Bäume haben

→ lineare Laufzeit, um von der Wurzel zum Blatt zu gelangen.

→ Warum wir von der Wurzel zum Blatt wollen, sehen wir erst später bei `removeMin()` und `add()`

Die Heap-Invariante ist lockerer als die BST-Invariante. Das heißt, wir können unsere **Heap-Struktur-Invariante** strikter als die AVL-Invariante gestalten.

Heap-Struktur-Invariante:

Ein Heap ist immer ein vollständiger Baum.

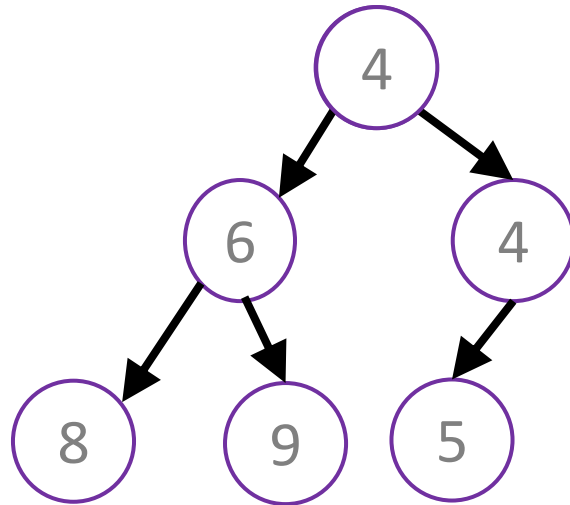
Heaps

Heap-Struktur-Invariante:

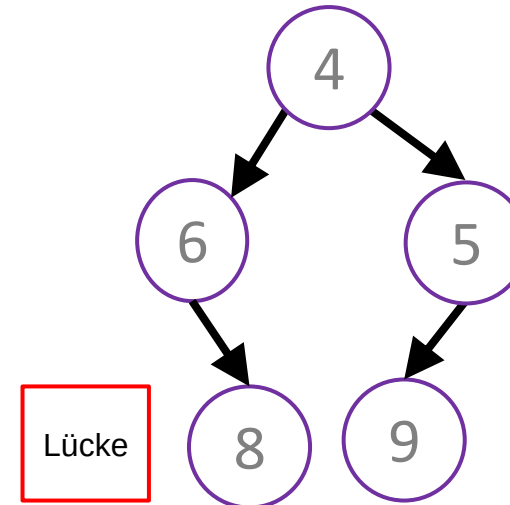
Ein Heap ist immer ein vollständiger Baum.

- Ein Baum ist vollständig, wenn:
 - Jede Ebene, außer möglicherweise der letzten (untersten), vollständig gefüllt ist.
 - Die unterste Ebene von links nach rechts gefüllt ist (keine „Lücke“)

Heap



Kein Heap



Binäre Heap-Invarianten

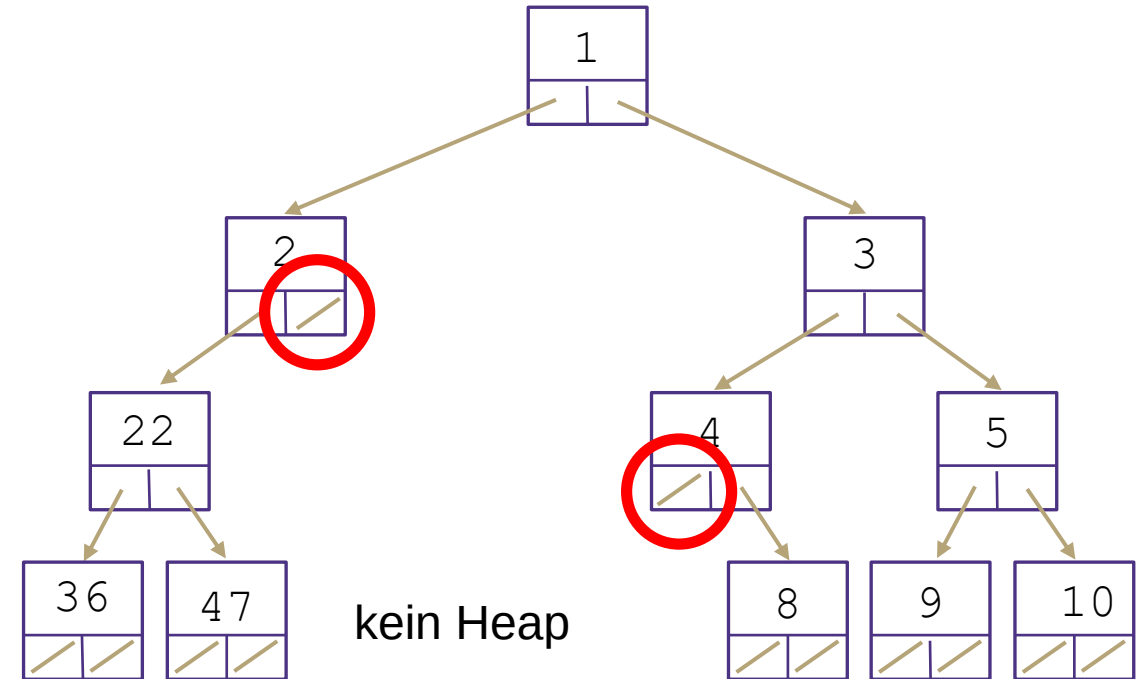
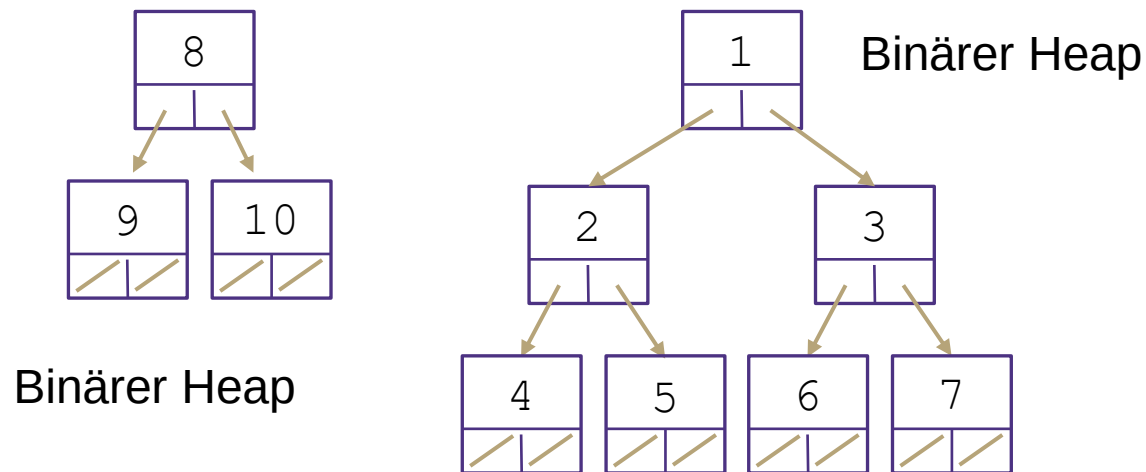
Übersicht

Eine Art von Heap ist ein **binärer Heap**.

1. Binärer Baum: jeder Knoten hat höchstens 2 Kinder
2. **Min-Heap-Invariante**: Der Schlüssel jedes Knotens ist kleiner (oder gleich) als die seiner Kinder

This is a big idea!
(heap invariants!)

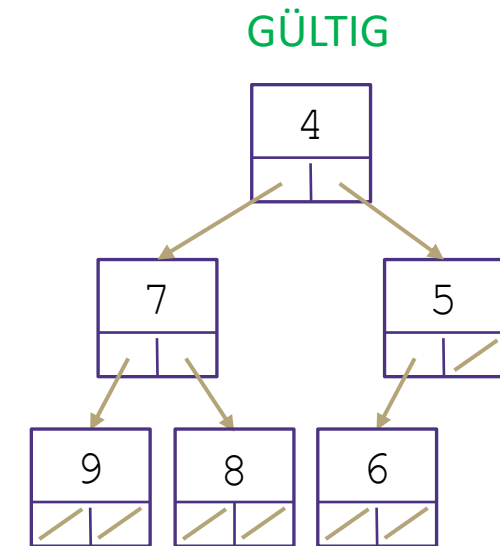
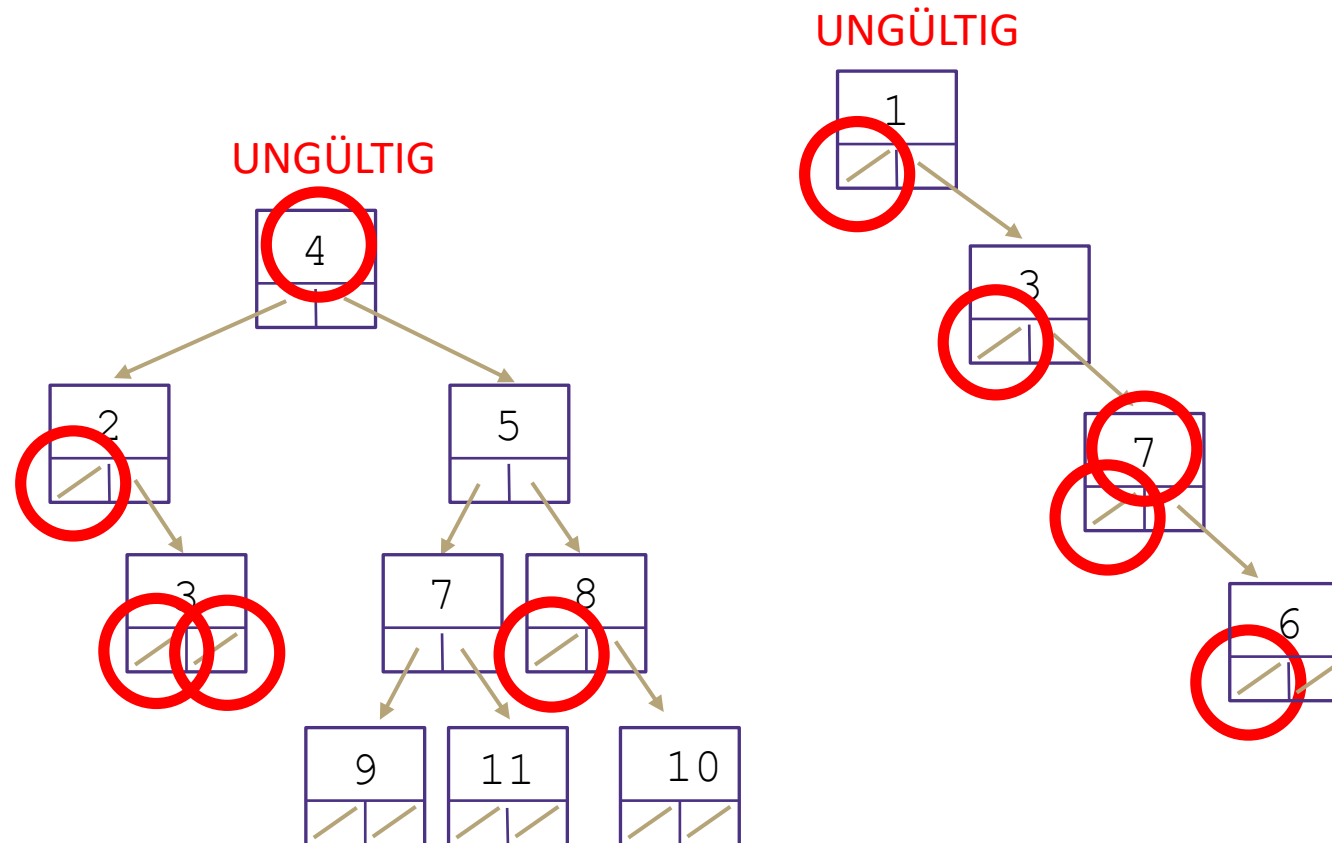
3. **Heap-Struktur-Invariante**: Jede Ebene ist „vollständig“, d. h. sie hat keine „Lücken“.
- Heaps werden von links nach rechts aufgefüllt



Selbstkontrolle - Sind diese Heaps gültig?

Binärer Heap Invarianten:

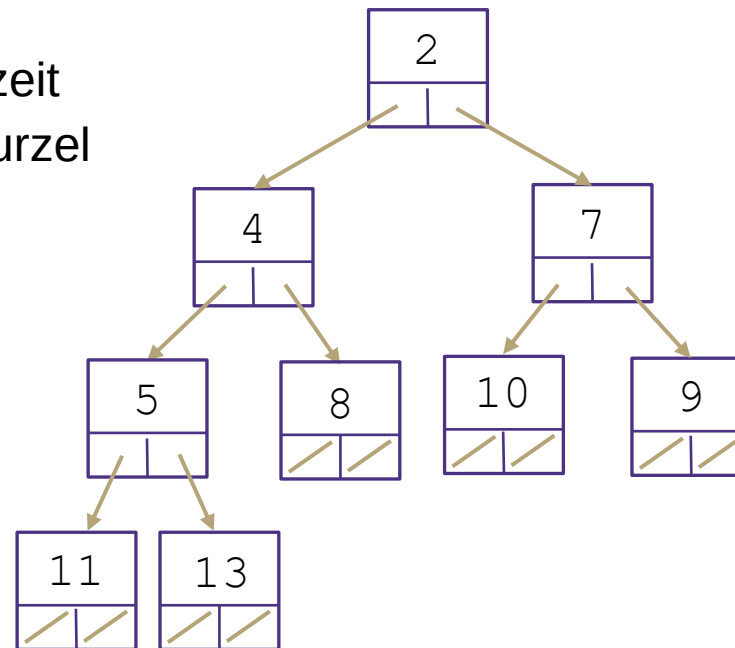
1. Binärer Baum
2. Heap-Invariante
3. Vollständig



Binärer Heap: Höhe

Ein binärer Heap begrenzt unsere Baumhöhe auf $\log(n)$, weil der Baum vollständig ist.

Das bedeutet, dass die Laufzeit für den Durchlauf von der Wurzel zum Blatt oder vom Blatt zur Wurzel in $\Theta(\log(n))$ liegt.



Fragen?

Dinge, über die wir gesprochen haben:

- Heap-Invarianten
- Binärer Heap
- Höhe des binären Heaps

Ihr Werkzeugkasten bis hierhin...

Abstrakte Datentypen

- Liste - Flexibilität, einfaches Verschieben von Elementen innerhalb der Struktur
- Stapel (Stack) - optimiert für First-in-Last-out-Reihenfolge
- Warteschlange - optimiert für die Reihenfolge First-in-First-out.
- Tabelle - speichert Schlüssel und Wert bei jedem Eintrag
- Prioritätswarteschlange - optimiert für Ausgabe der höchsten Priorität zuerst

Implementierung von Datenstrukturen

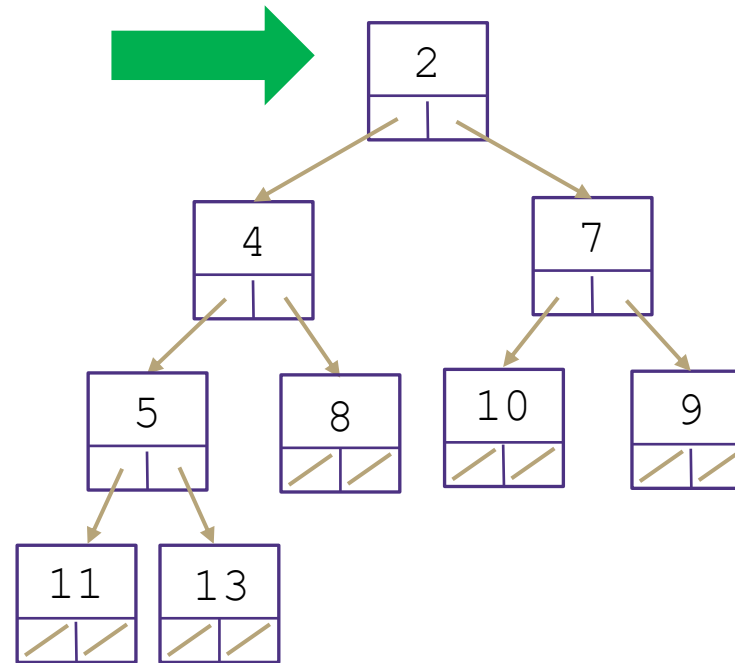
- Array - leicht ein Element über Index zu suchen, schwer umzuordnen
- Verkettete Liste - schwer ein Element zu suchen, leicht umzuordnen
- Hashtabelle - konstante Zeit zum Suchen, keine Ordnung der Daten
- Binärer Suchbaum - effizientes Suchen, Möglichkeit eines schlechten Worst Case
- AVL-Baum - effizientes Suchen, schützt vor dem Worst Case, schwer zu implementieren
- Heap - effizient für Min oder Max Werte

Inhalt: Prioritätswarteschlange / Heaps

- Prioritätswarteschlange als ADT
- Prioritätswarteschlange Implementierungen mit aktuellem Werkzeugkasten
- Binärer Heap: Idee + Invarianten
- **Binärer Heap: Methoden**
 - peekMin
 - removeMin
 - add
- Prioritätswarteschlange implementiert als binärer Heap
- Binärer Heap
 - Floyd's Heap Construction Algorithmus

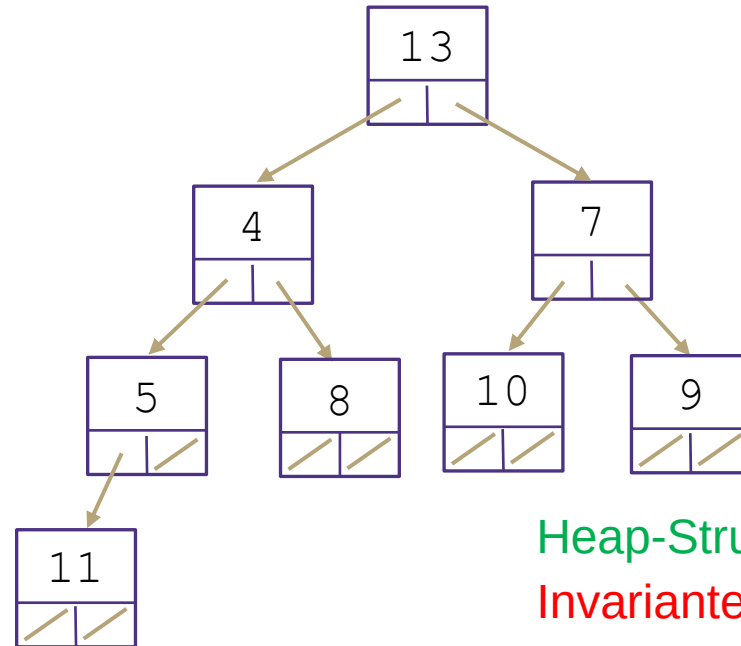
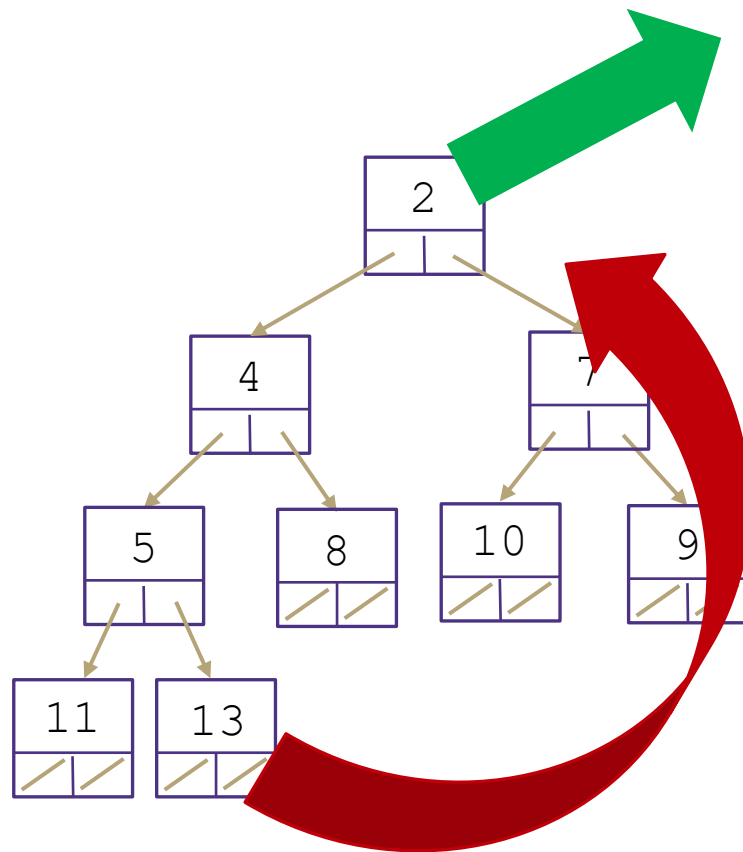
Binärer Heap: peekMin() Implementierung

Laufzeit: $\Theta(1)$



Binärer Heap: removeMin() Implementierung

1. Rückgabe des Min-Schlüssel-Wert-Paars
2. Ersetzen durch den untersten rechten Knoten



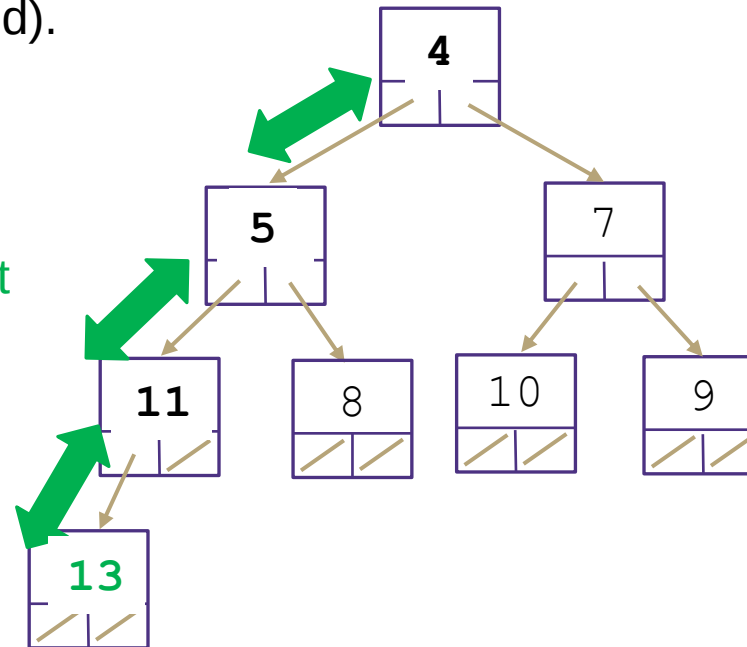
Heap-Struktur-Invariante wiederhergestellt, Heap-Invariante gebrochen

Binärer Heap: removeMin() percolateDown

3.) percolateDown() (nach unten versickern)

Rekursiver Tausch des Elternteils mit dem kleinsten Kind bis das Elternteil kleiner ist als beide Kinder (oder wir bei einem Blatt sind).

Heap-Struktur-Invariante
wiederhergestellt,
Heap-Invariante wiederhergestellt



Praktikum: heapifyDown()

Wie lange ist die Worst Case Laufzeit?

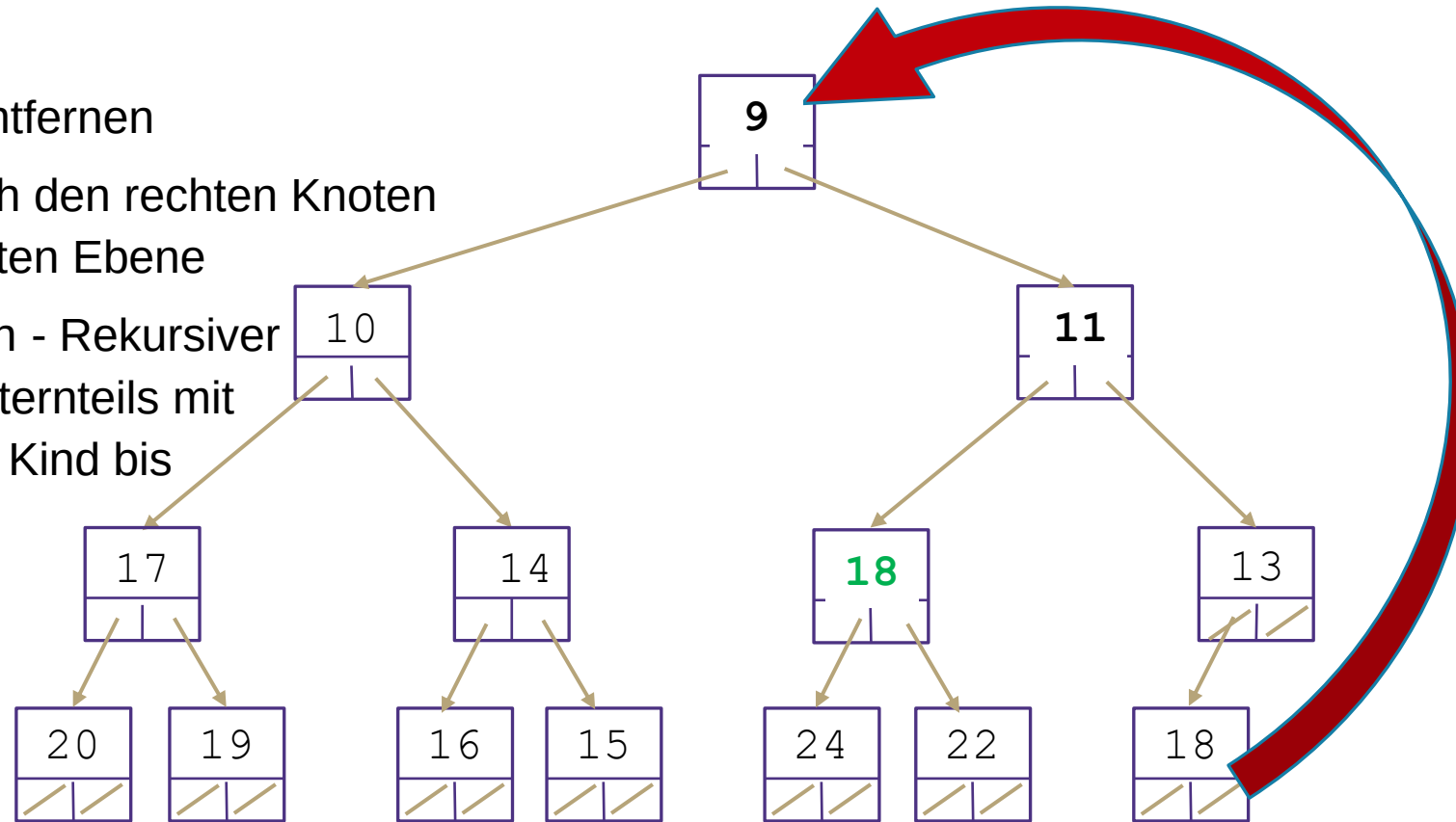
Wir müssen:

- Den untersten rechten Knoten finden
 - Nicht trivial – Laufzeit unbekannt
- An die Wurzel verschieben
- PercolateDown, bis Heap-Invariante wiederhergestellt ist (wie oft?)

Aus diesem Grund wollen wir die Höhe des Baums klein halten! Die Höhe dieser Baumstrukturen (BST, AVL, Heaps) korreliert direkt mit den Worst Case-Laufzeiten

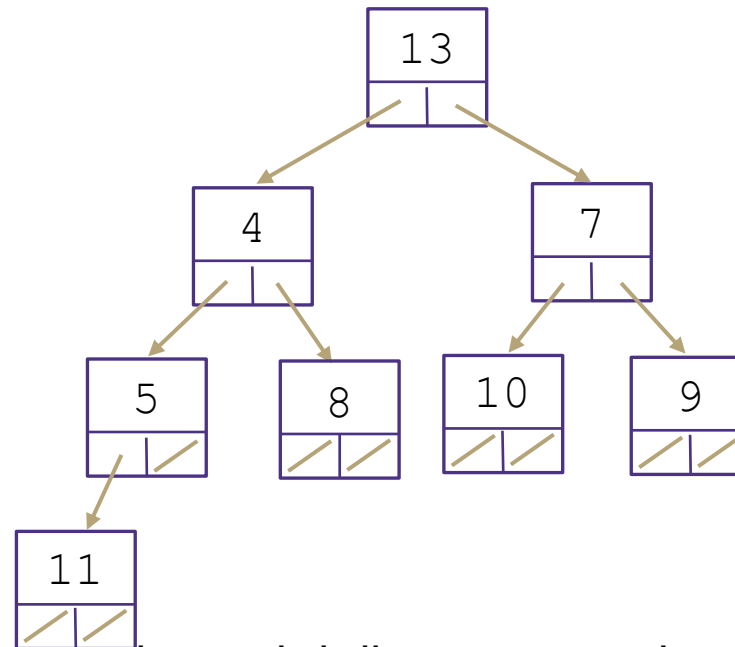
Übung: removeMin()

1. Min-Knoten entfernen
2. Ersetzen durch den rechten Knoten auf der untersten Ebene
3. percolateDown - Rekursiver Tausch des Elternteils mit dem kleinsten Kind bis das Elternteil kleiner ist als beide Kinder (oder wir bei einem Blatt sind).



Binärer Heap: removeMin() percolateDown

Warum tauscht percolateDown mit dem kleinsten Kind und nicht mit einem beliebigen Kind?



Wenn wir 13 und 7 vertauschen, wird die Heap-Invariante nicht wiederhergestellt!

7 ist größer als 4 (es ist nicht das kleinste Kind!), so dass die Heap-Invariante verletzt wird.

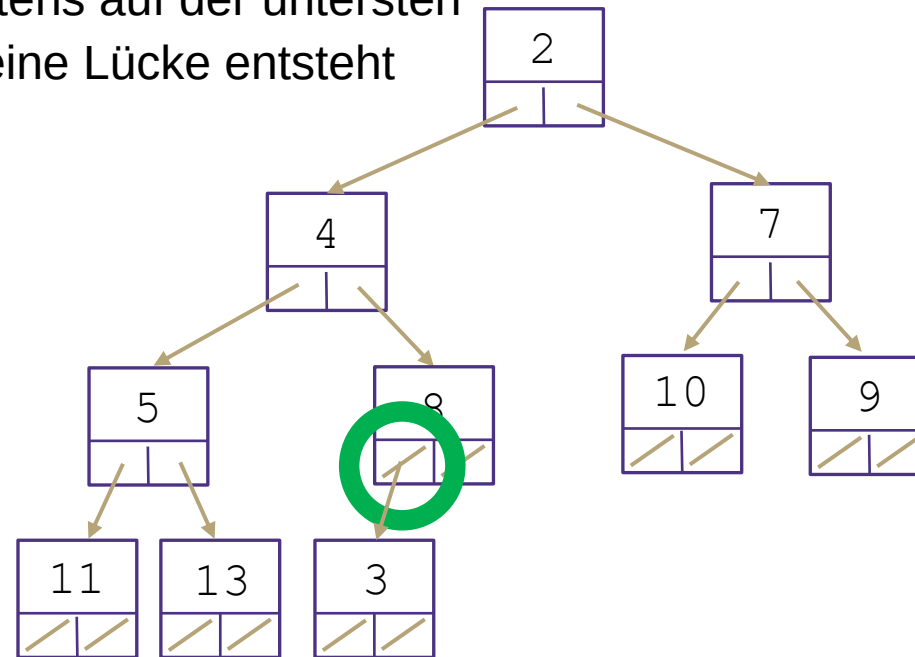
Fragen?

- Übungsblatt 6: Aufgabe 8

Binärer Heap: add() Implementierung

add() Methode:

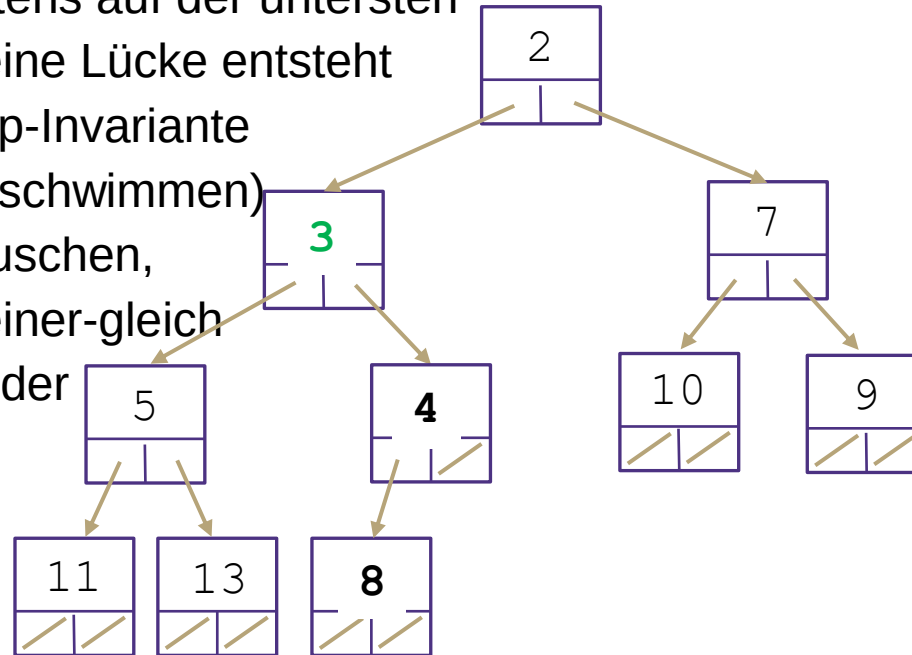
- Einfügen eines Knotens auf der untersten Ebene, ohne dass eine Lücke entsteht



Binärer Heap: add() Implementierung

add() Algorithmus:

- Einfügen eines Knotens auf der untersten Ebene, ohne dass eine Lücke entsteht
- Reparieren der Heap-Invariante durch percolateUp (schwimmen)
- d.h. mit Elternteil tauschen, bis das Elternteil kleiner-gleich ist als man selbst (oder man die Wurzel ist).



- Die Laufzeit im Worst Case ist ähnlich wie bei removeMin und percolateDown - möglicherweise müssen $\log(n)$ Swaps durchgeführt werden, so dass die Laufzeit im Worst Case $\Theta(\log(n))$ beträgt.

Übung: Erstellen eines Min-Heap

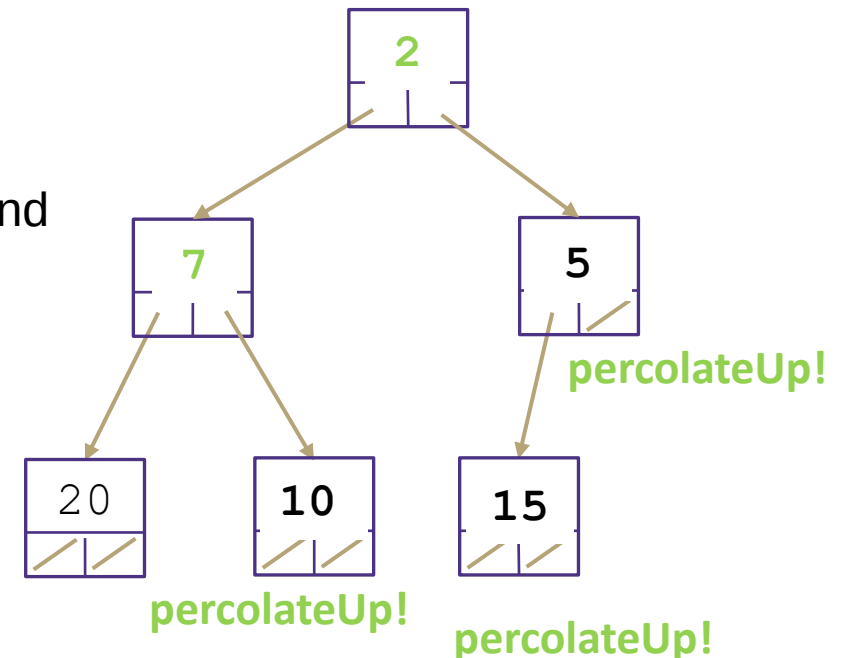
Min Binary Heap Invariants

1. **Binary Tree** – each node has at most 2 children
2. **Min Heap** – each node's children are larger than itself
3. **Level Complete** - new nodes are added from left to right completely filling each level before creating a new one

Konstruieren Sie einen binären Min-Heap, indem Sie die folgenden Schlüssel in dieser Reihenfolge hinzufügen: 5, 10, 15, 20, 7, 2

add() Algorithmus:

- Einfügen eines Knotens auf der untersten Ebene, um sicherzustellen, dass keine Lücken vorhanden sind
- Reparieren der Heap-Invariante durch percolateUp (schwimmen)
- d.h. mit Elternteil tauschen, bis das Elternteil kleiner-gleich ist als man selbst (oder man die Wurzel ist).



Heap: Laufzeiten der Methoden `remove()` und `add()`

`removeMin()`:

- Wurzelknoten entfernen - $\Theta(1)$
- Untersten rechten Knoten finden und zum Wurzelknoten machen - $\Theta(\log n)$ – nicht trivial
- `PercolateDown`, um Heap-Invariante wiederherzustellen - $\Theta(\log n)$

`add()`:

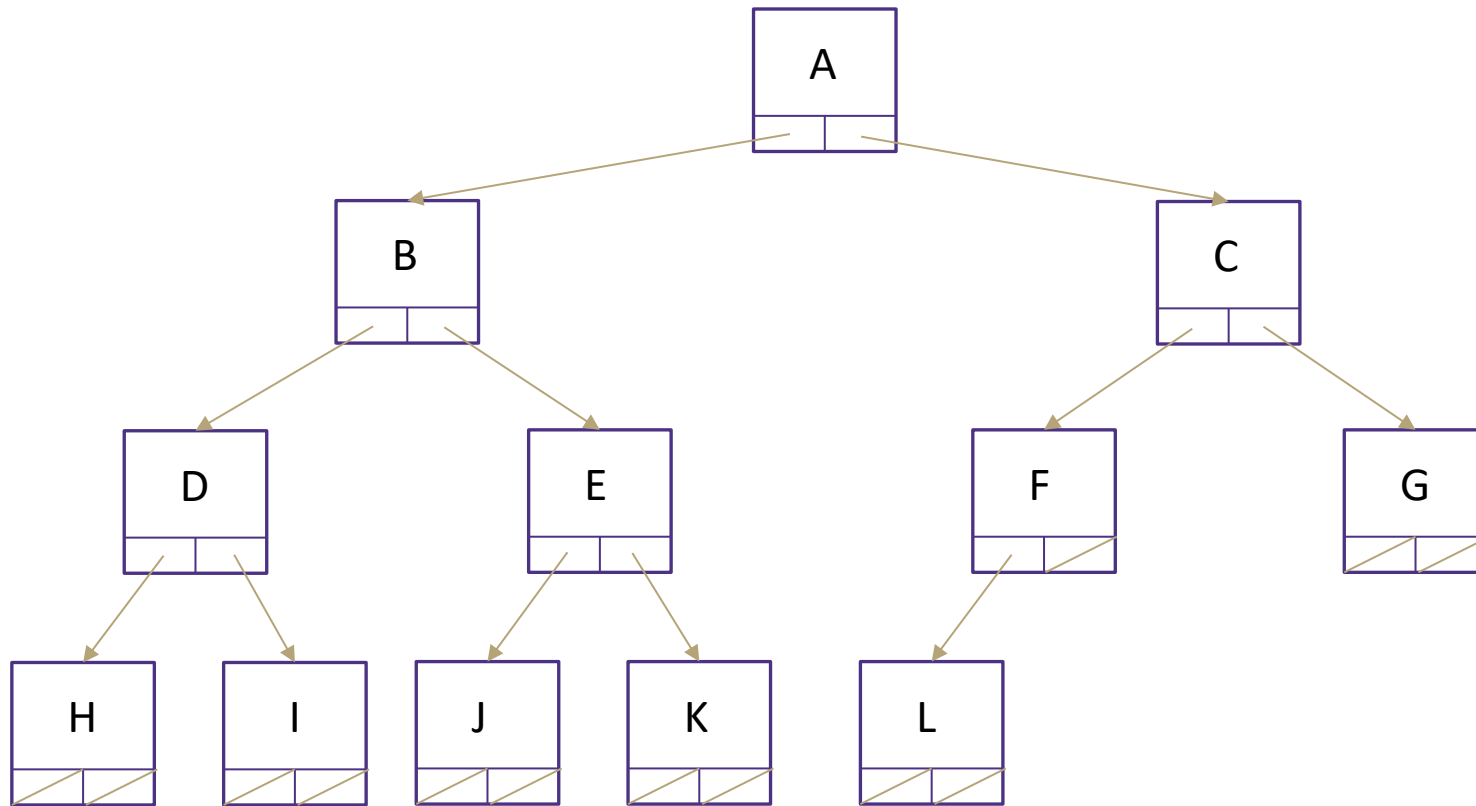
- Einfügen eines neuen Knotens an der nächsten freien Stelle - $\Theta(\log n)$ – nicht trivial
- `PercolateUp`, um Heap-Invariante wiederherzustellen - $\Theta(\log n)$

Den untersten rechten Knoten/die nächst freie Stelle zu finden, ist der schwierige Part

Für vollständige Bäume geht es in $\Theta(\log n)$, aber eher schwierig

Es gibt einen viel besseren Weg!

Heap implementiert durch ein Array



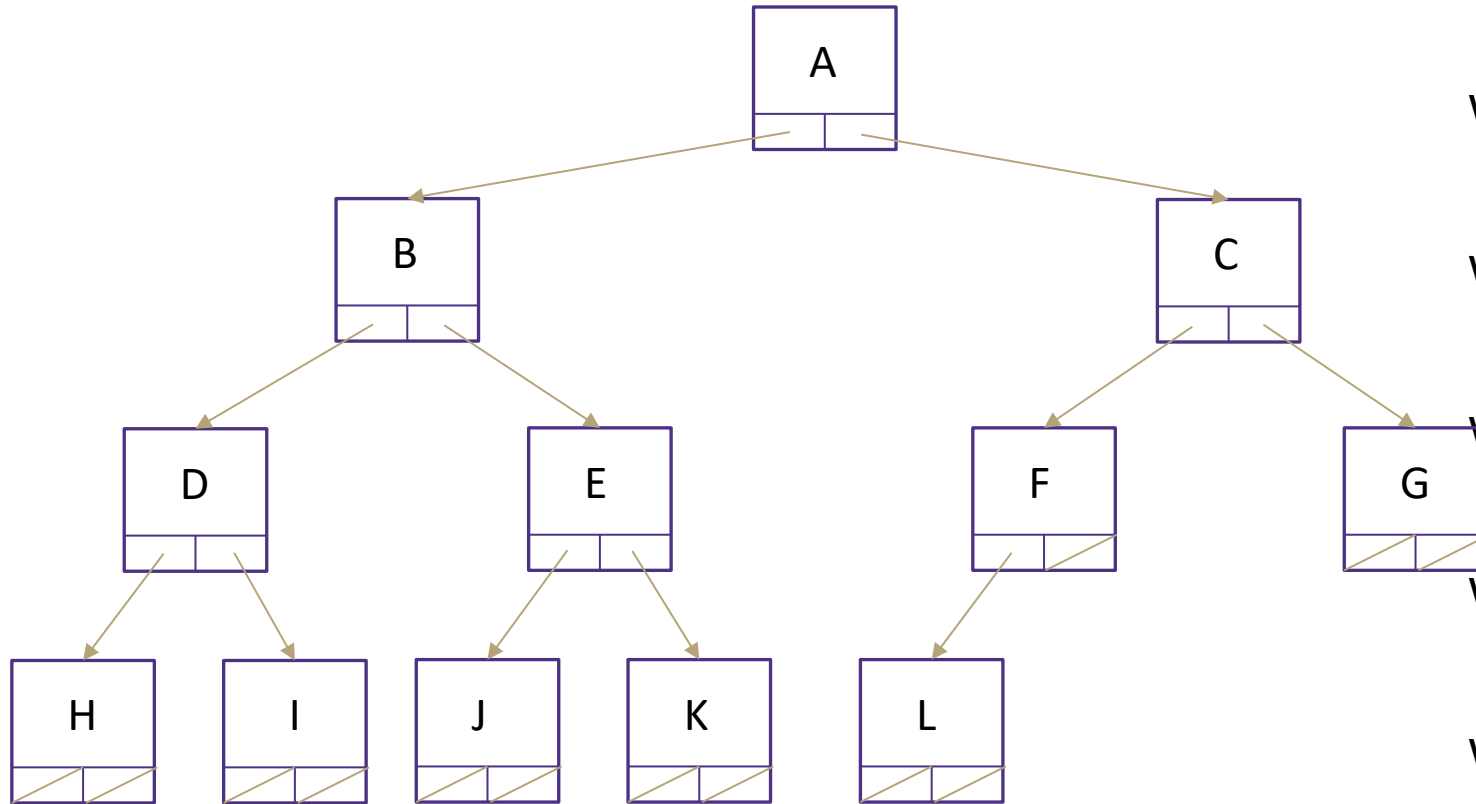
Wir bilden unsere Binärbaum-Darstellung eines Heaps als Array-Implementierung ab, bei der wir das Array in der Reihenfolge der Ebenen von links nach rechts auffüllen.

Die Array-Implementierung eines Heaps ist das, was man tatsächlich implementiert, aber die Baumdarstellung ist die Art und Weise, wie man es sich konzeptionell vorstellt. Alles, was wir über die Baumdarstellung besprochen haben, ist nach wie vor richtig!

Füllen Sie das Array Ebene für Ebene von links nach rechts

0	1	2	3	4	5	6	7	8	9	10	11	12	13
A	B	C	D	E	F	G	H	I	J	K	L		

Heap implementiert durch ein Array



Wie findet man den minimalen Knoten?

$$\text{peekMin}() = \text{arr}[0]$$

Wie findet man den letzten Knoten?

$$\text{lastNode}() = \text{arr}[\text{size} - 1]$$

Wie findet man die nächste freie Stelle?

$$\text{openSpace}() = \text{arr}[\text{size}]$$

Wie findet man das linke Kind eines Knotens i?

$$\text{leftChild}(i) = 2i + 1$$

Wie findet man das rechte Kind eines Knotens?

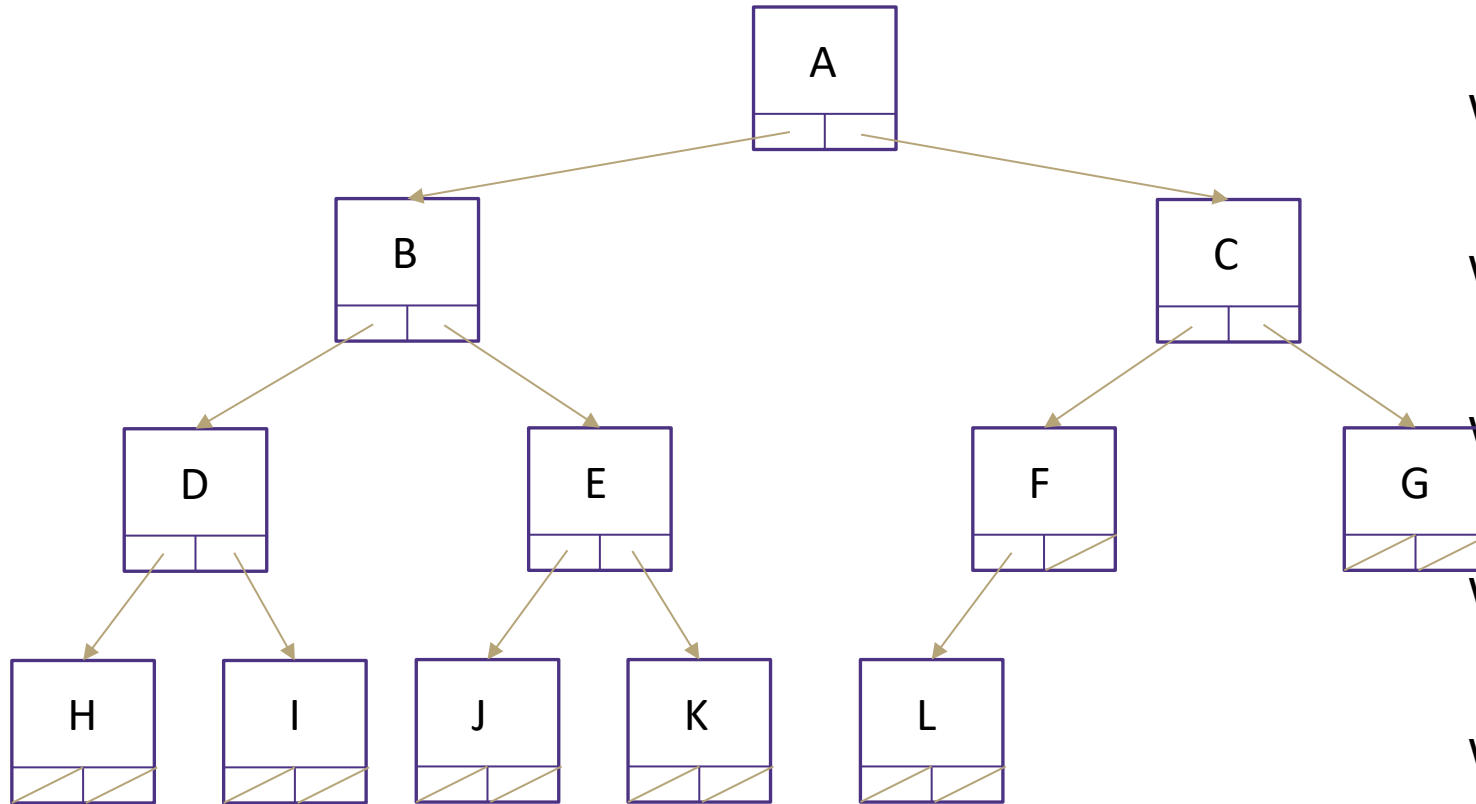
$$\text{rightChild}(i) = 2i + 2$$

Wie findet man das Elternteil eines Knotens i?

$$\text{parent}(i) = \left\lfloor \frac{i - 1}{2} \right\rfloor$$

0	1	2	3	4	5	6	7	8	9	10	11	12	13
A	B	C	D	E	F	G	H	I	J	K	L		

Heap implementiert durch ein Array



Wie findet man den minimalen Knoten?

$$\text{peekMin}() = \text{arr}[1]$$

Wie findet man den letzten Knoten?

$$\text{lastNode}() = \text{arr}[\text{size}]$$

Wie findet man die nächste freie Stelle?

$$\text{openSpace}() = \text{arr}[\text{size} + 1]$$

Wie findet man das linke Kind eines Knotens?

$$\text{leftChild}(i) = 2i$$

Wie findet man das rechte Kind eines Knotens?

$$\text{rightChild}(i) = 2i + 1$$

Wie findet man das Elternteil eines Knotens?

$$\text{parent}(i) = \left\lfloor \frac{i}{2} \right\rfloor$$

0	1	2	3	4	5	6	7	8	9	10	11	12	13
/	A	B	C	D	E	F	G	H	I	J	K	L	

Heap: Laufzeiten der Methoden `remove()` und `add()`

Array-Implementierung

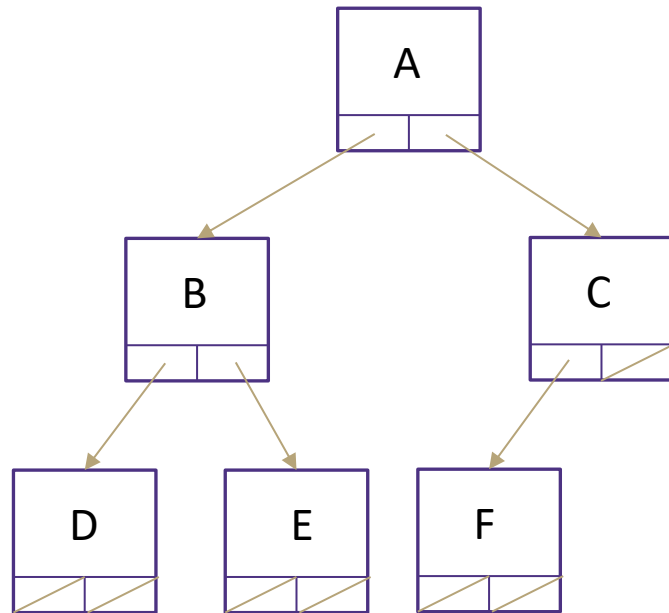
`removeMin()`:

- Wurzelknoten entfernen - $\Theta(1)$
- Untersten rechten Knoten finden und zum Wurzelknoten machen - $\Theta(1)$ – trivial
- `PercolateDown`, um Heap-Invariante wiederherzustellen - $\Theta(\log n)$

`add()`:

- Einfügen eines neuen Knotens an der nächsten freien Stelle - $\Theta(1)$ – trivial
- `PercolateUp`, um Heap-Invariante wiederherzustellen - $\Theta(\log n)$

Heap Implementierung: Laufzeiten



Implementierung	add	removeMin	peek
Array-basierter Heap	worst: $\Theta(\log n)$ in-practice: $\Theta(1)$	worst: $\Theta(\log n)$ in-practice: $\Theta(\log n)$	$\Theta(1)$

Wir haben das asymptotische Worst Case-Verhalten von AVL-Bäumen erreicht.

Aber wir sind sogar noch besser!

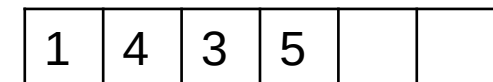
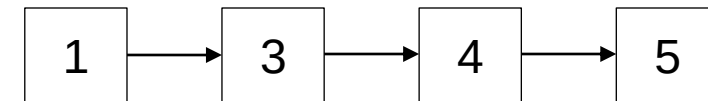
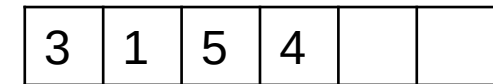
- Die konstanten Faktoren für Array-Zugriffe sind besser.
- Der Heap kann um einen konstanten Faktor kürzer sein, weil die Heap-Struktur-Invariante strenger ist.
- Der „In der Praxis“ Fall für add ist wirklich gut.
- Ein Heap ist VIEL einfacher zu implementieren.

Inhalt: Prioritätswarteschlange / Heaps

- Prioritätswarteschlange als ADT
- Prioritätswarteschlange Implementierungen mit aktuellem Werkzeugkasten
- Binärer Heap: Idee + Invarianten
- Binärer Heap: Methoden
 - peekMin
 - removeMin
 - add
- **Prioritätswarteschlange implementiert als binärer Heap**
- Binärer Heap
 - Floyd's Heap Construction Algorithmus

Implementierung von Prioritätswarteschlangen: Versuch II

Implementierung	add	removeMin	peekMin
Unsortiertes Array	$\theta(1)$	$\theta(n)$	$\theta(n)$
Verkettete Liste (sortiert)	$\theta(n)$	$\theta(1)$	$\theta(1)$
AVL-Baum	$\theta(\log n)$	$\theta(\log n)$	$\theta(\log n)$
Heap als Array	$\theta(\log n)$ In der Praxis: $\theta(1)$	$\theta(\log n)$	$\theta(1)$



Sind Heaps immer besser? AVL-Baum vs. Heap

Heap

Vorteile:

- Array: schnellerer Speicherzugriff
- `add()` in $\Theta(1)$ in der Praxis

Nachteile:

- `containsKey()/get()` in $\Theta(n)$, da lineare Suche

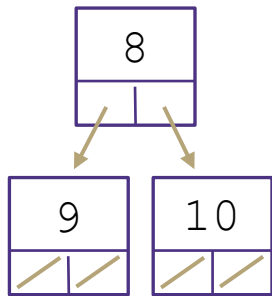
AVL-Baum

Vorteile:

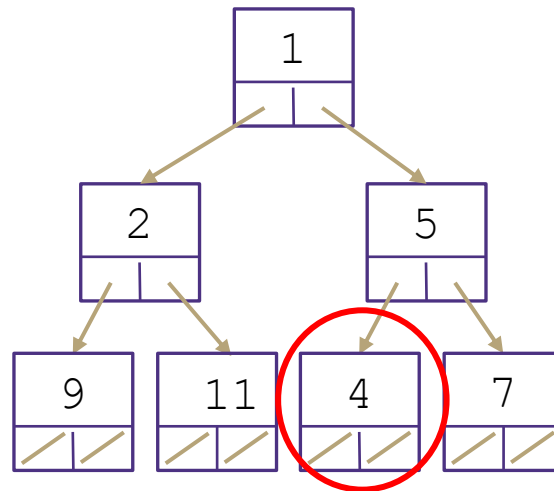
- absolut sortiert
 - In-Order Traversierung in $O(n)$
- `containsKey()/get()` in $\Theta(\log(n))$

Übung: Binäre Min-Heaps

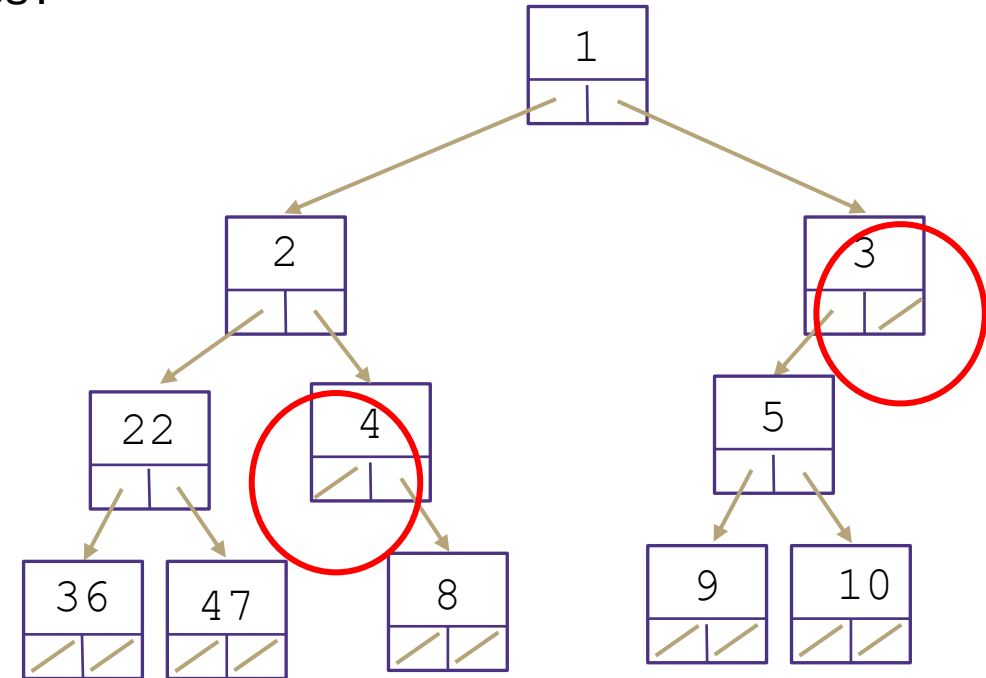
Sind die folgenden Bäume gültige binäre Min-Heaps?



Gültig



Ungültig



Ungültig

Fragen?

- Übungsblatt 6: Aufgabe 6 und 2 a, b

Inhalt: Prioritätswarteschlange / Heaps

- Prioritätswarteschlange als ADT
- Prioritätswarteschlange Implementierungen mit aktuellem Werkzeugkasten
- Binärer Heap: Idee + Invarianten
- Binärer Heap: Methoden
 - peekMin
 - removeMin
 - add
- Prioritätswarteschlange implementiert als binärer Heap
- Binärer Heap
 - **Floyd's Heap Construction Algorithmus**

Wiederholung

Übung: Erstellen eines Min-Heap

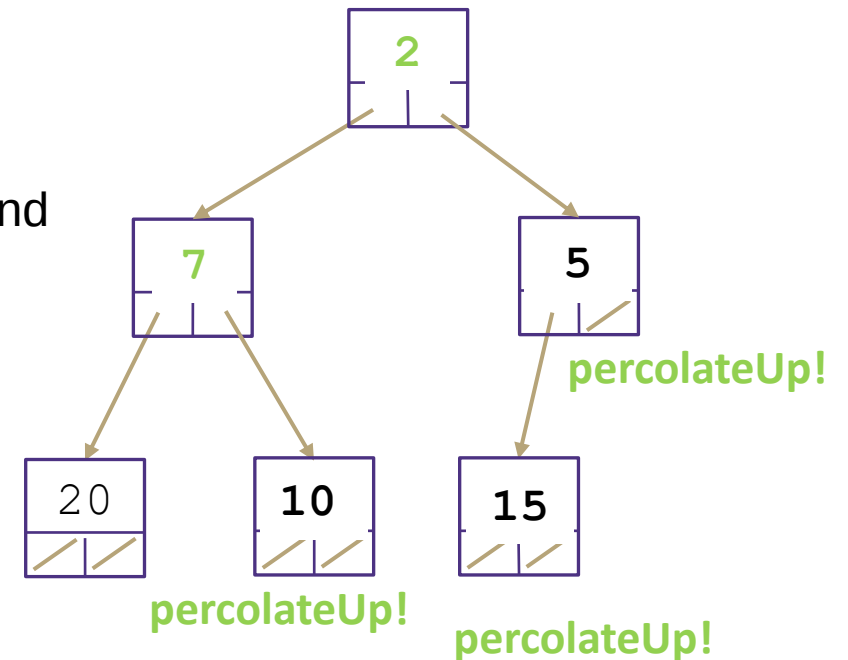
Min Binary Heap Invariants

1. **Binary Tree** – each node has at most 2 children
2. **Min Heap** – each node's children are larger than itself
3. **Level Complete** - new nodes are added from left to right completely filling each level before creating a new one

Konstruieren Sie einen binären Min-Heap, indem Sie die folgenden Schlüssel in dieser Reihenfolge hinzufügen: 5, 10, 15, 20, 7, 2

add() Algorithmus:

- Einfügen eines Knotens auf der untersten Ebene, um sicherzustellen, dass keine Lücken vorhanden sind
- Reparieren der Heap-Invariante durch percolateUp (schwimmen)
- d.h. mit Elternteil tauschen, bis das Elternteil kleiner-gleich ist als man selbst (oder man die Wurzel ist).



Heap: Erstellen eines Heaps (BuildHeap)

BuildHeap(Schlüssel $k_1, \dots, k_n$) – Gegeben n Schlüssel, erstelle Heap, der diese n Schlüssel enthält

Versuch 1:

- Rufe `add()` n -mal auf. (so wie vorhin in Übung – Folie 29)
- Worst Case Laufzeit?
 - n Aufrufe, jeder hat eine Worst Case Laufzeit von $O(\log n)$. $\rightarrow O(n \log n)$

Versuch 2: (Floyd's Heap Construction Algorithmus – Floyd's heapify)

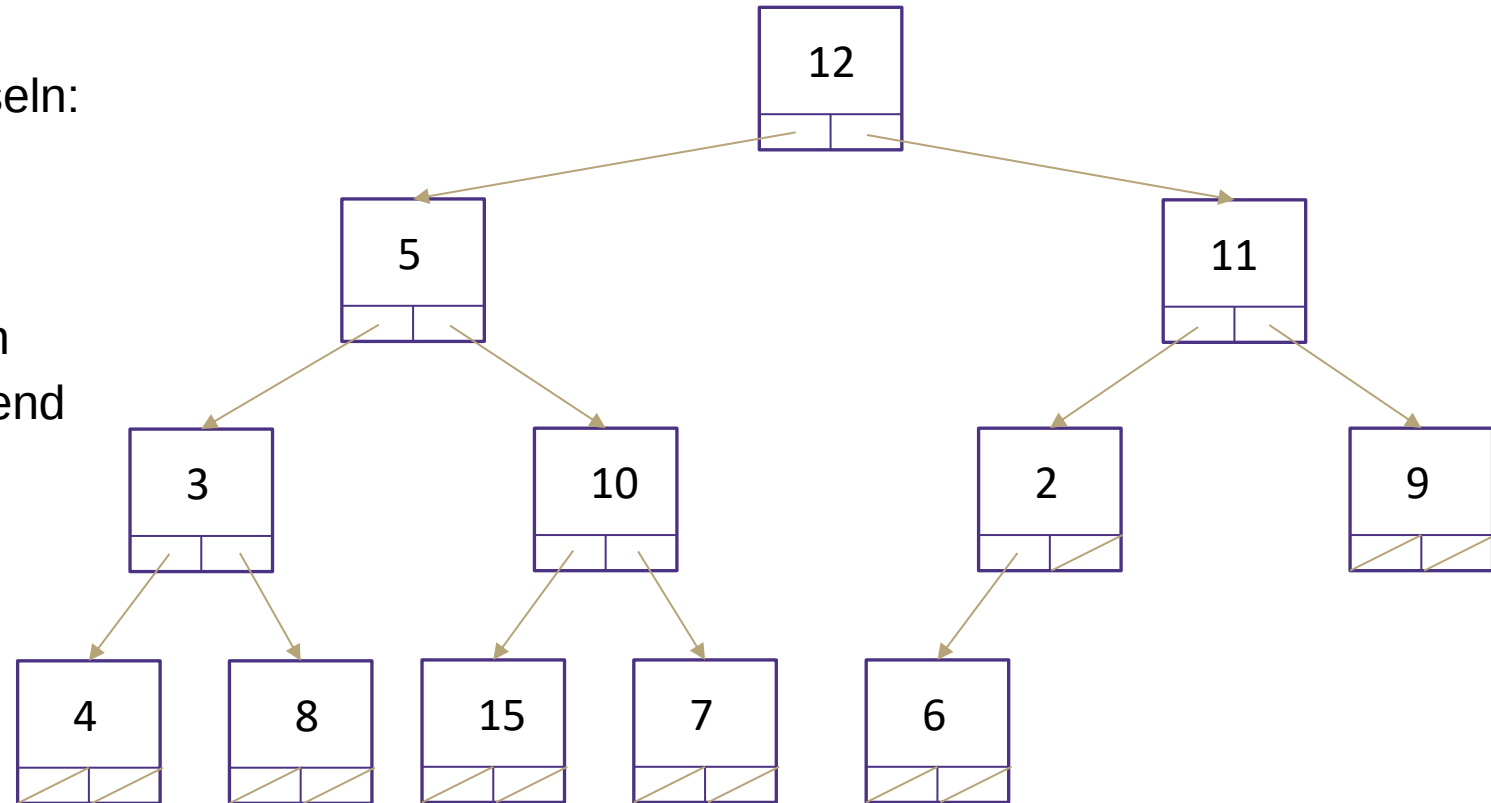
- Füge alle Schlüssel in das Array ein (ohne Ordnung)
- Gehe alle Schlüssel durch und rufe `PercolateDown` auf, um Heap-Invariante zu erfüllen
- Wir rufen `PercolateDown` auf und nicht `PercolateUp`, da in letzter Ebene des Baumes mehr Knoten sind, als in Spitze des Baumes. Annahme, dass dadurch weniger Schlüssel bewegt und auch nur eine „konstante Distanz“ durch den Baum gehen müssen.

Floyd's Heap Construction Algorithmus

Erstelle einen Heap mit den Schlüsseln:

12, 5, 11, 3, 10, 2, 9, 4, 8, 15, 7, 6

1. Füge alle Schlüssel in Array ein
2. percolateDown(parent) beginnend mit letztem Index



0	1	2	3	4	5	6	7	8	9	10	11	12	13
12	5	11	3	10	2	9	4	8	15	7	6		

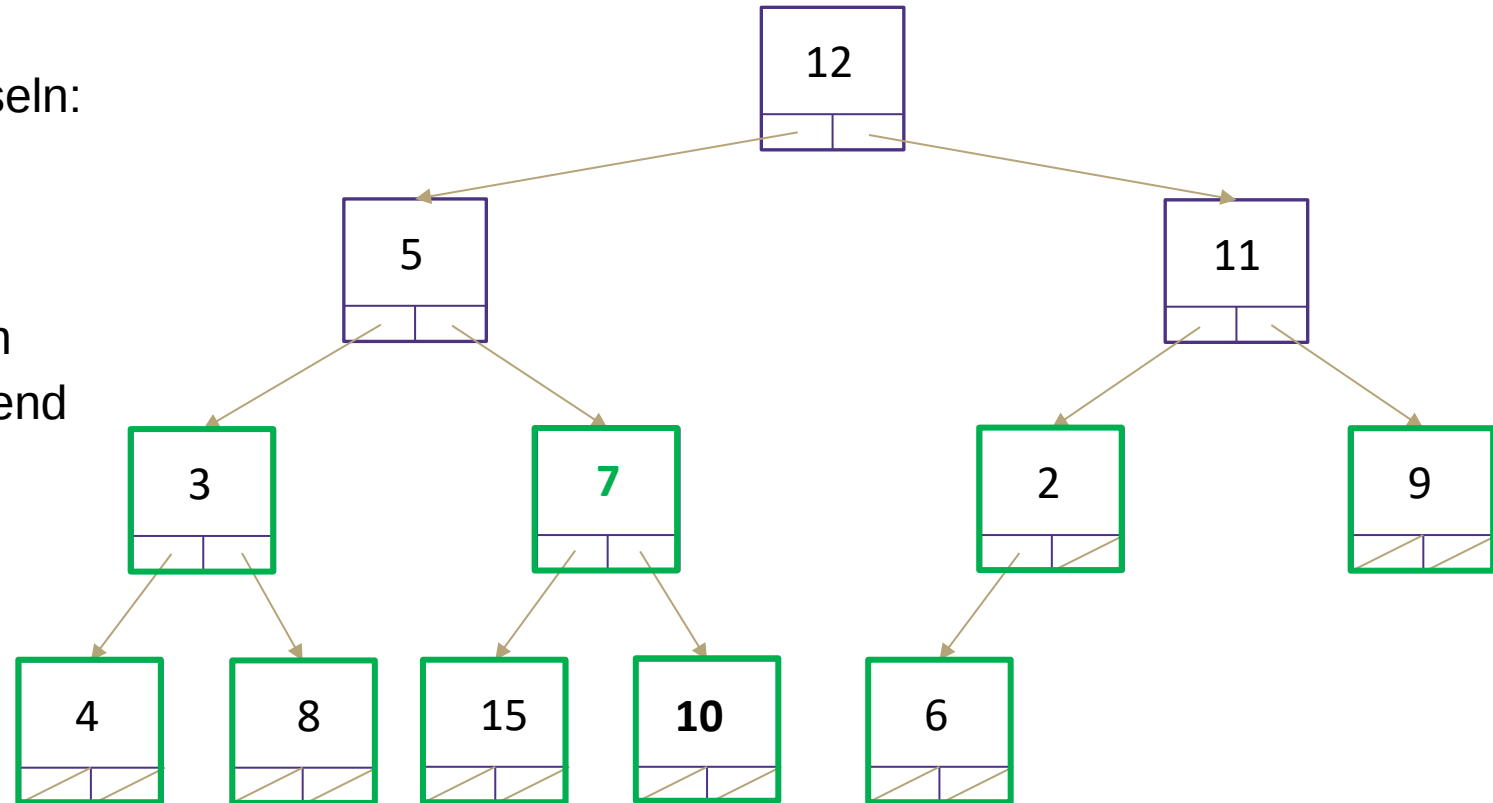


Floyd's Heap Construction Algorithmus

Erstelle einen Heap mit den Schlüsseln:

12, 5, 11, 3, 10, 2, 9, 4, 8, 15, 7, 6

1. Füge alle Schlüssel in Array ein
2. percolateDown(parent) beginnend mit letztem Index
 1. percolateDown Ebene 4
 2. percolateDown Ebene 3



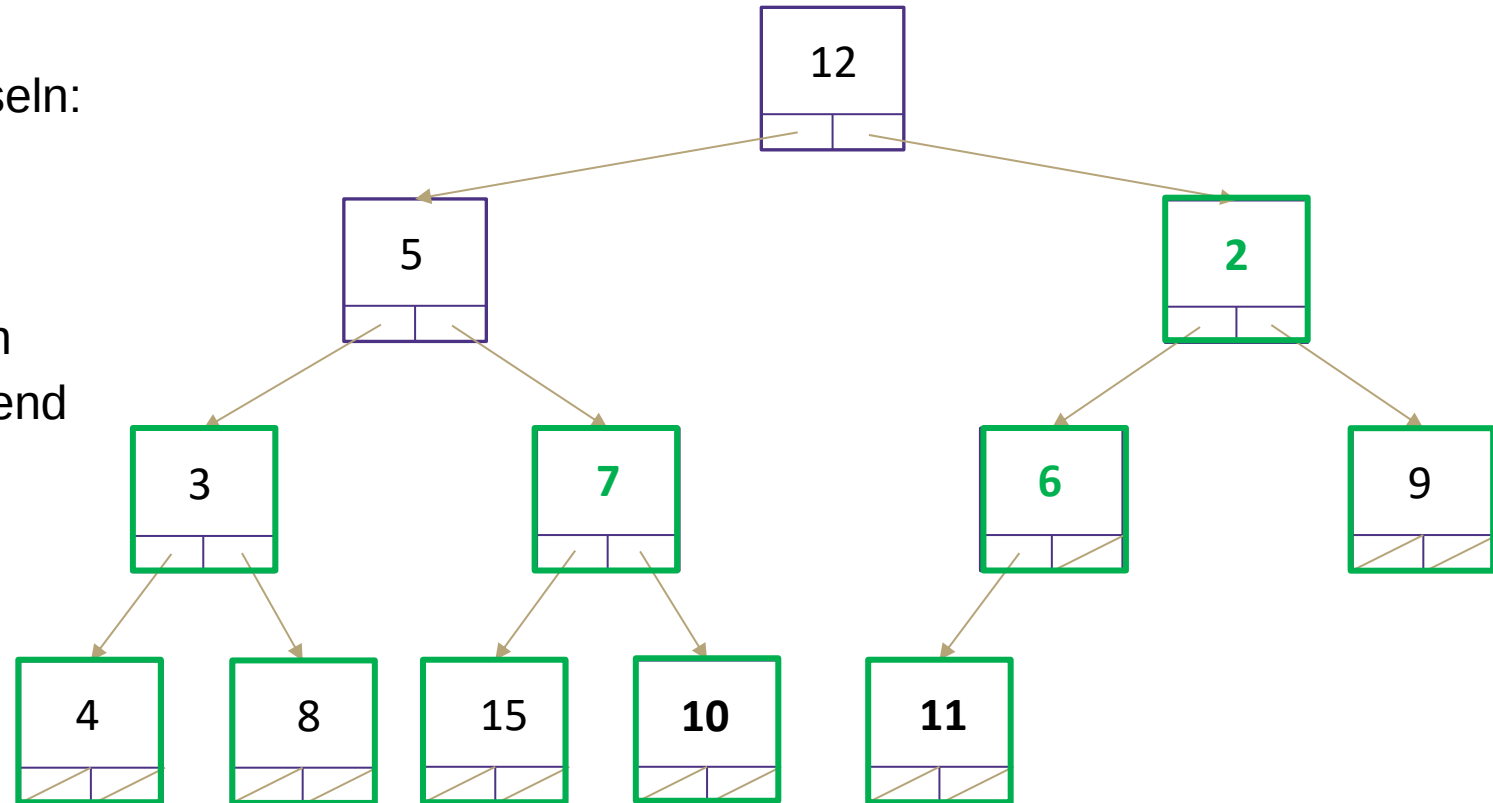
0	1	2	3	4	5	6	7	8	9	10	11	12	13
12	5	11	3	7	2	9	4	8	15	10	6		

Floyd's Heap Construction Algorithmus

Erstelle einen Heap mit den Schlüsseln:

12, 5, 11, 3, 10, 2, 9, 4, 8, 15, 7, 6

1. Füge alle Schlüssel in Array ein
2. percolateDown(parent) beginnend mit letztem Index
 1. percolateDown Ebene 4
 2. percolateDown Ebene 3
 3. percolateDown Ebene 2



0	1	2	3	4	5	6	7	8	9	10	11	12	13
12	5	11	3	7	2	9	4	8	15	10	6		

26.08.2026

Seite 57

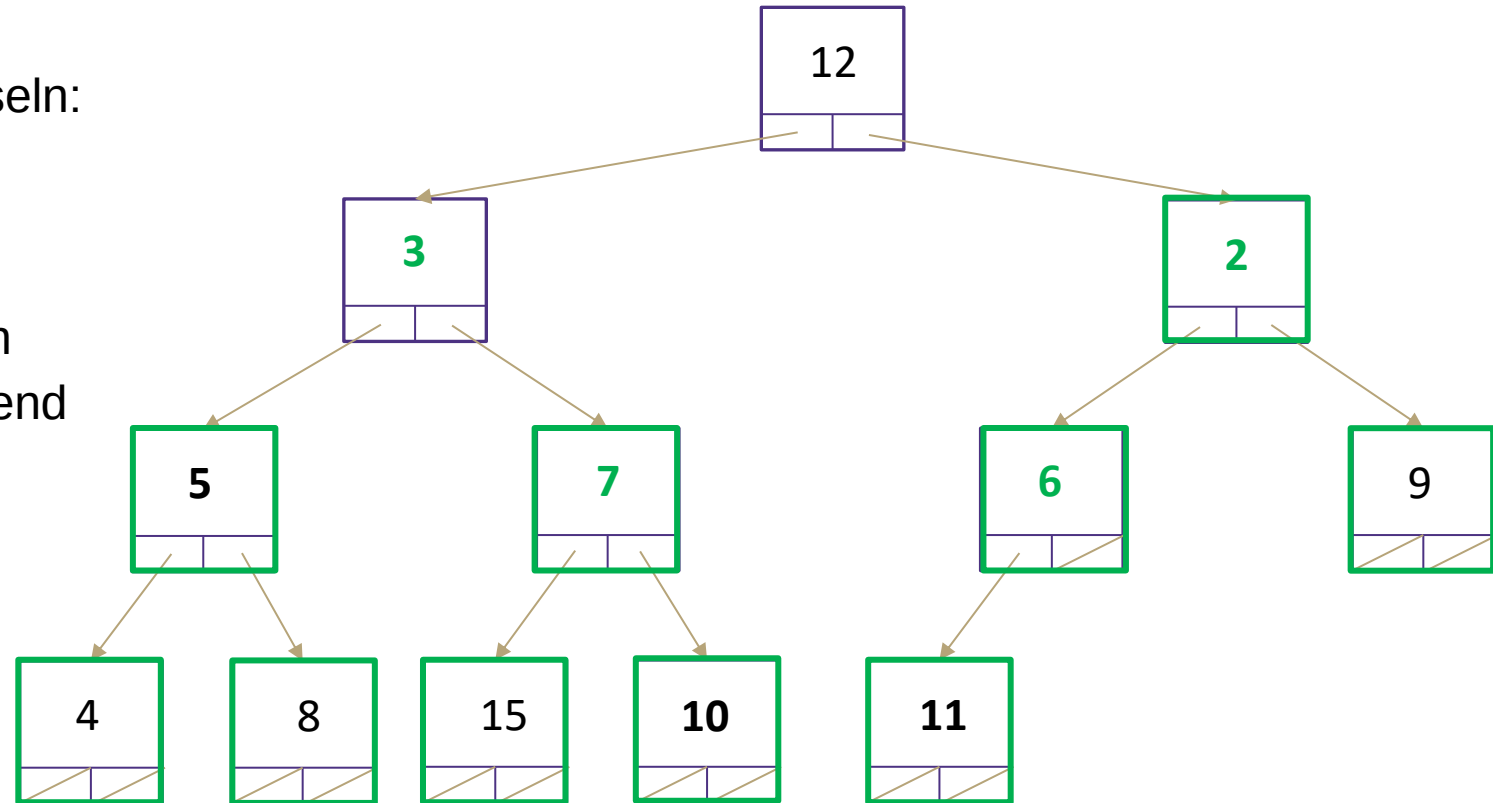
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Floyd's Heap Construction Algorithmus

Erstelle einen Heap mit den Schlüsseln:

12, 5, 11, 3, 10, 2, 9, 4, 8, 15, 7, 6

1. Füge alle Schlüssel in Array ein
2. percolateDown(parent) beginnend mit letztem Index
 1. percolateDown Ebene 4
 2. percolateDown Ebene 3
 3. percolateDown Ebene 2



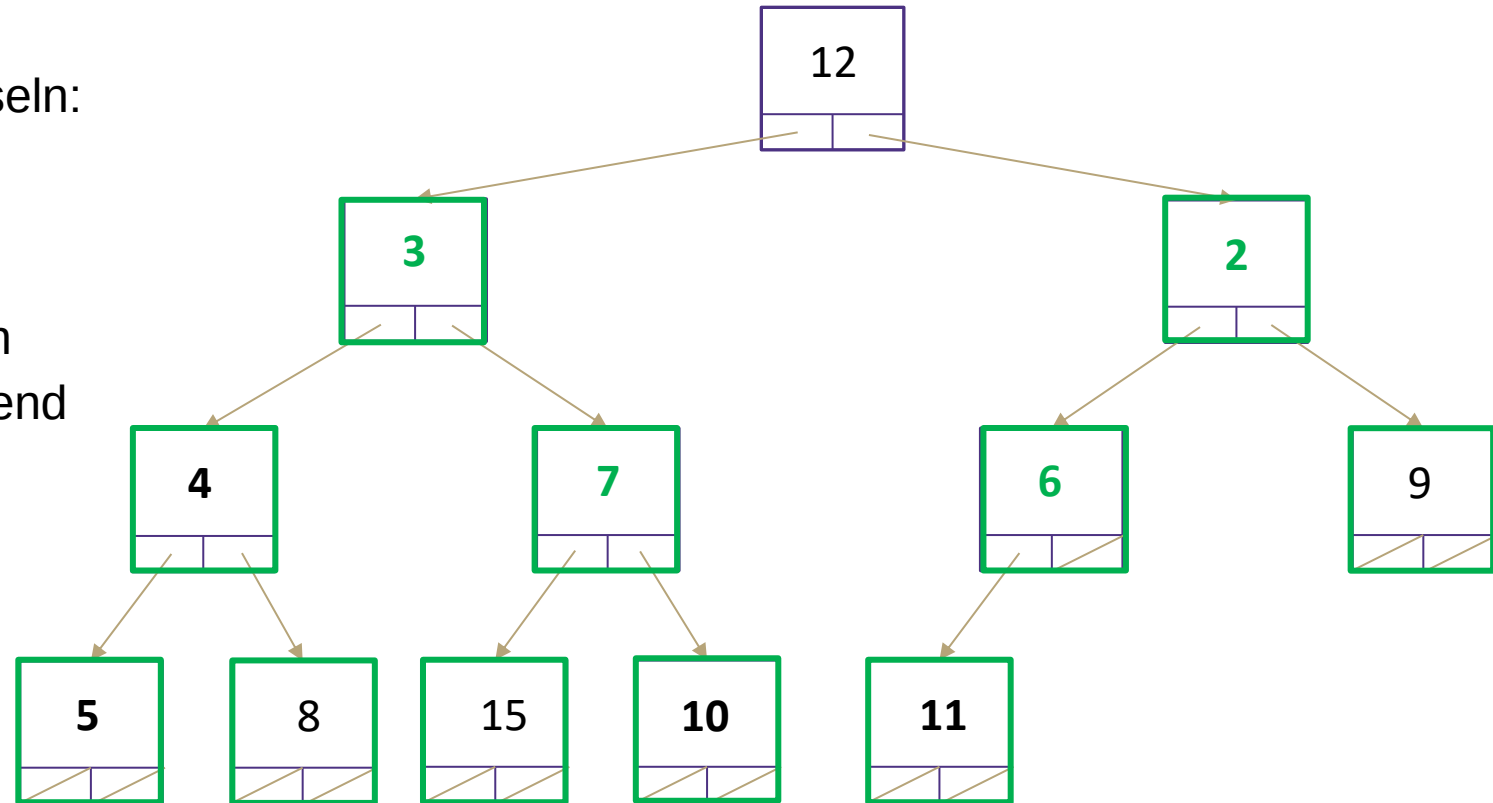
0	1	2	3	4	5	6	7	8	9	10	11	12	13
12	5	2	3	7	6	9	4	8	15	10	11		

Floyd's Heap Construction Algorithmus

Erstelle einen Heap mit den Schlüsseln:

12, 5, 11, 3, 10, 2, 9, 4, 8, 15, 7, 6

1. Füge alle Schlüssel in Array ein
2. percolateDown(parent) beginnend mit letztem Index
 1. percolateDown Ebene 4
 2. percolateDown Ebene 3
 3. percolateDown Ebene 2



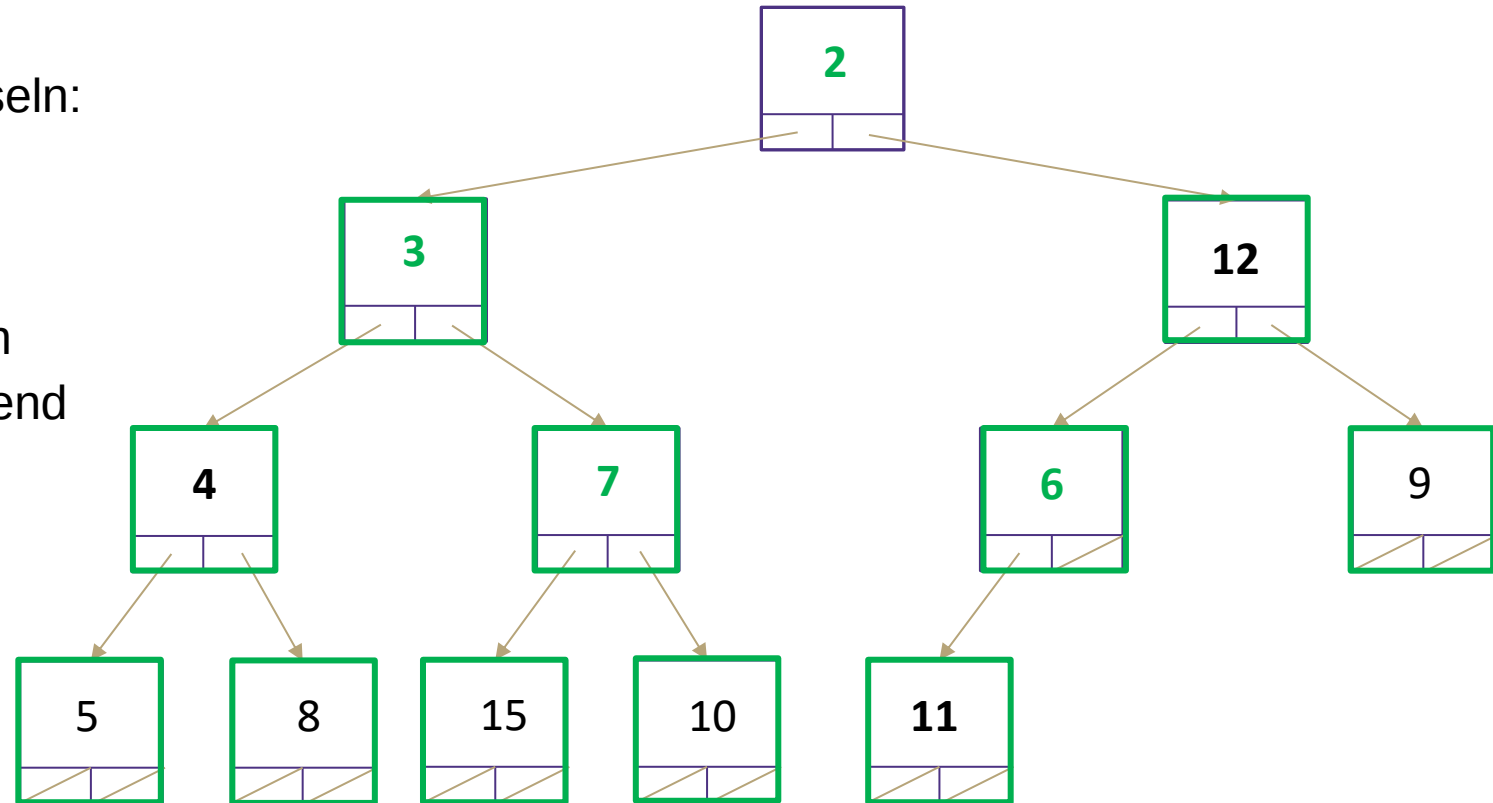
0	1	2	3	4	5	6	7	8	9	10	11	12	13
12	3	2	4	7	6	9	5	8	15	10	11		

Floyd's Heap Construction Algorithmus

Erstelle einen Heap mit den Schlüsseln:

12, 5, 11, 3, 10, 2, 9, 4, 8, 15, 7, 6

1. Füge alle Schlüssel in Array ein
2. percolateDown(parent) beginnend mit letztem Index
 1. percolateDown Ebene 4
 2. percolateDown Ebene 3
 3. percolateDown Ebene 2
 4. percolateDown Ebene 1



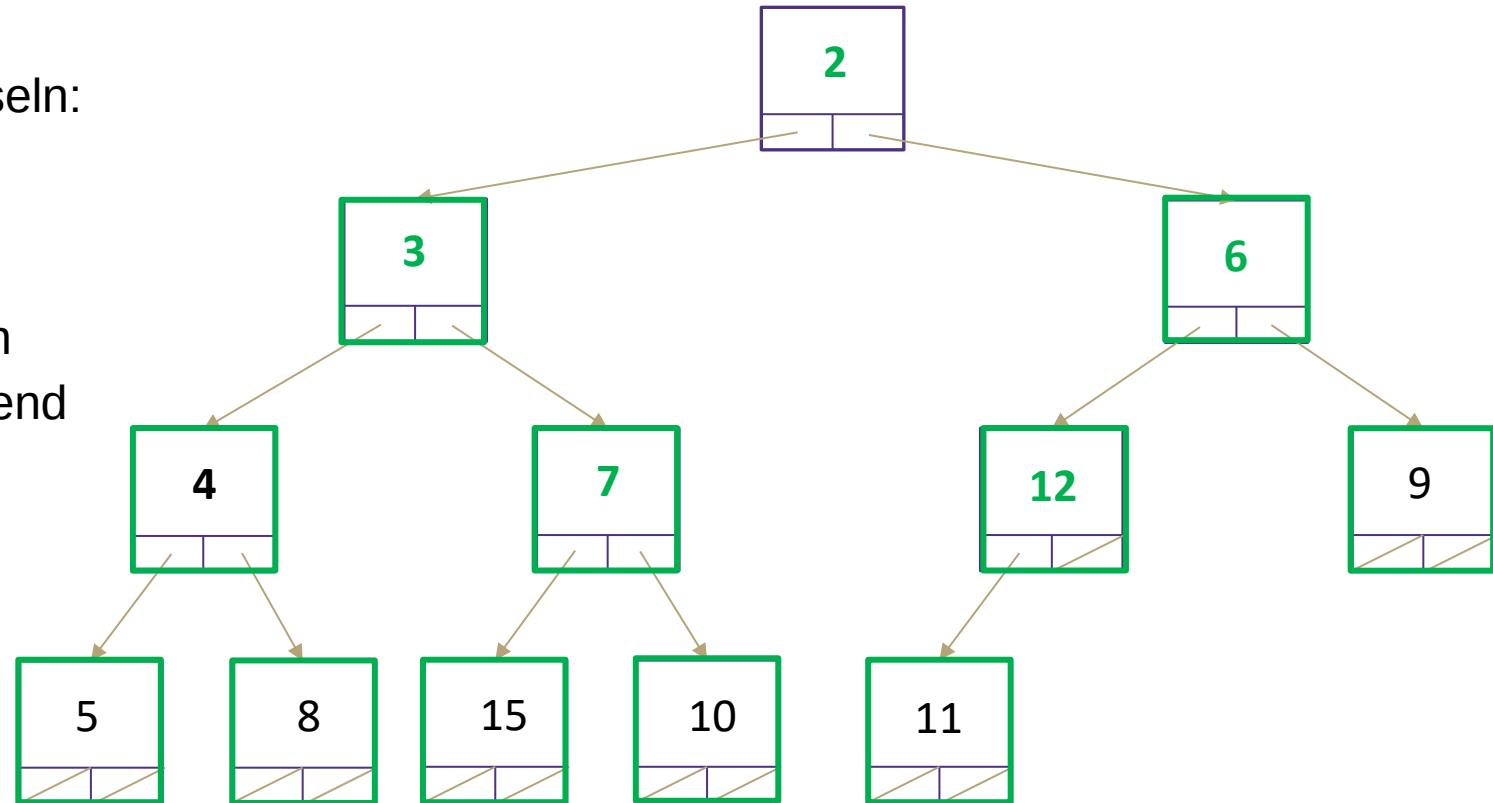
	0	1	2	3	4	5	6	7	8	9	10	11	12	13
26.08.2026	12	3	2	4	7	6	9	5	8	15	10	11		

Floyd's Heap Construction Algorithmus

Erstelle einen Heap mit den Schlüsseln:

12, 5, 11, 3, 10, 2, 9, 4, 8, 15, 7, 6

1. Füge alle Schlüssel in Array ein
2. percolateDown(parent) beginnend mit letztem Index
 1. percolateDown Ebene 4
 2. percolateDown Ebene 3
 3. percolateDown Ebene 2
 4. percolateDown Ebene 1



0	1	2	3	4	5	6	7	8	9	10	11	12	13
2	3	6	4	7	12	9	5	8	15	10	11		

26.08.2026

Seite 61

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

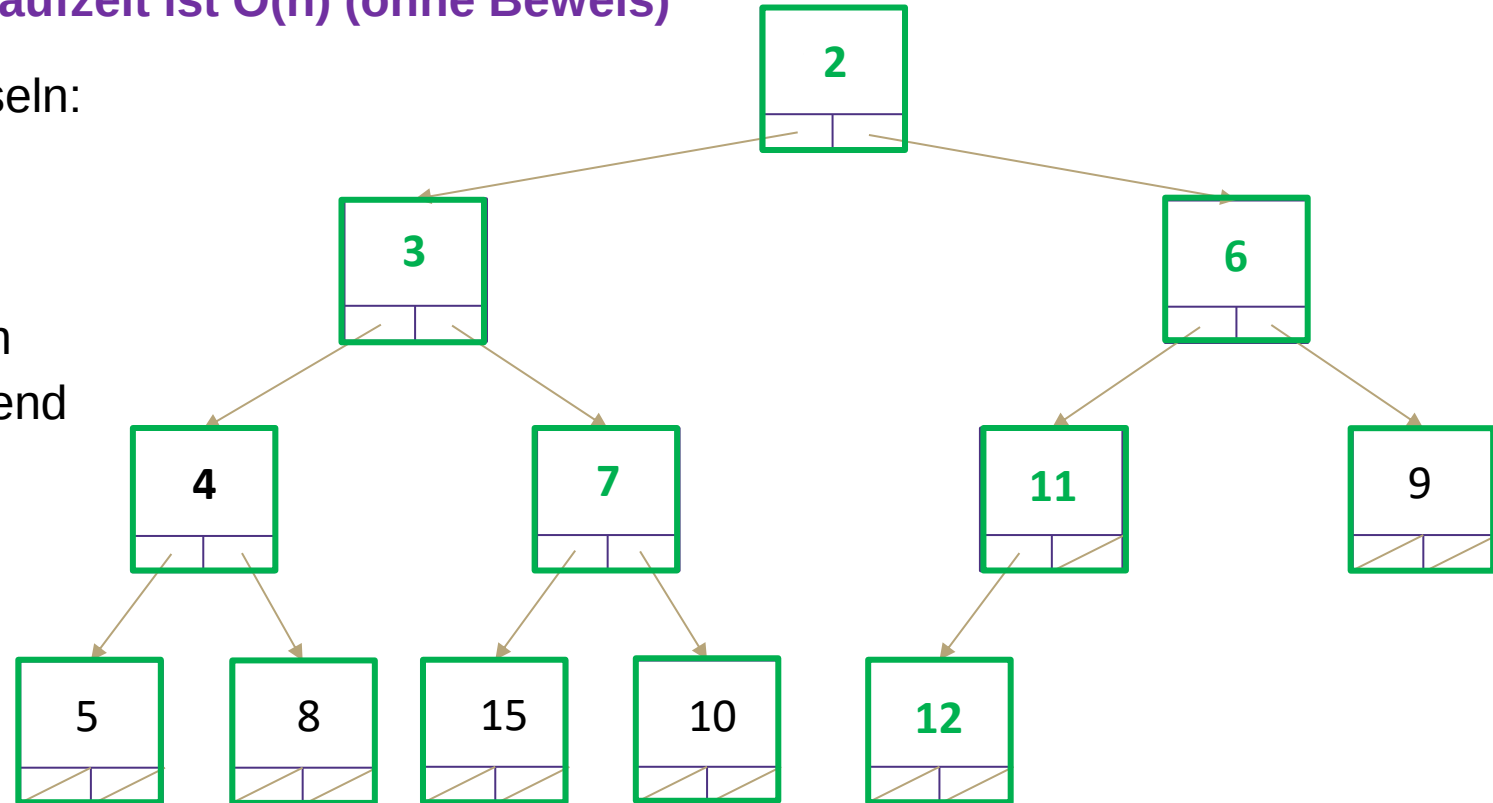
Floyd's Heap Construction Algorithmus

Laufzeit ist $O(n)$ (ohne Beweis)

Erstelle einen Heap mit den Schlüsseln:

12, 5, 11, 3, 10, 2, 9, 4, 8, 15, 7, 6

1. Füge alle Schlüssel in Array ein
2. percolateDown(parent) beginnend mit letztem Index
 1. percolateDown Ebene 4
 2. percolateDown Ebene 3
 3. percolateDown Ebene 2
 4. percolateDown Ebene 1



0	1	2	3	4	5	6	7	8	9	10	11	12	13
2	3	6	4	7	11	9	5	8	15	10	12		

26.08.2026

Seite 62

Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Zusammenfassung: Prioritätswarteschlange / Heaps

- ADT: Prioritätswarteschlange – „Most Important out First“
- Binärer Heap: Idee + Invarianten
 - Man erhält Minimum bzw. Maximum in $\Theta(1)$
- Prioritätswarteschlangen werden als binäre Heaps implementiert
- Binärer Heap: Methoden
 - peekMin – $\Theta(1)$
 - removeMin – $\Theta(\log n)$
 - add – $\Theta(\log n)$ – In der Praxis sogar $\Theta(1)$
 - Floyd's Heap Construction Algorithmus – $\Theta(n)$
 - Besser als add() n-mal aufzurufen

Lernziele von heute und Fragen zur Überprüfung der Lernziele

Die Studierenden können den ADT Prioritätswarteschlange in Anwendungen nutzen, indem sie diese als Heap implementieren, um später effiziente Implementierungen von Prioritätswarteschlangen in Anwendungen nutzen zu können.

s. Übungsblatt 6 und Praktikum

Nächste Vorlesung und Fragen

- Lernraum III: Graphen
- Fragen?